CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY

ODDANCHATRAM - 624 619. DINDIGUL DT, TAMILNADU.

NAME :_____

REGISTER NO.

DEPARTMENT : **INFORMATION TECHNOLOGY**

COURSE : **B.Tech, AIDS**

YEAR / SEMESTER : **IV/VII**

SUBJECT NAME : **SALESFORCE DEVELOPER**

SUBJECT CODE : **SB8067**

BONAFIDE CERTIFICATE

Certified that this project report "WHATNEXT VISION MOTORS: SHAPING THE FUTURE OF MOBILITY WITH INNOVATION AND EXCELLENCE" is the bona-fide work of "DURGA DEVI" who carried out the project under my supervision

Dr. C. SUNDAR, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Assistant Professor
Department of ADS,
Christian College of Engg & Tech,
Oddanchatram-624619.

Mrs. V. SUMATHI, M.E.
SUPERVISOR
Assistant Professor
Department of ADS,
Christian College of Engg & Tech,
Oddanchatram-624619.

Submitted for the University Practical examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

S.NO	TITLE	PAGE NO
1	BONAFIDE CERTIFICATE	1
2	INTRODUCTION	3
3	IDEATION PHASE	4
4	REQUIREMENT ANALYSIS PHASE	7
5	PROJECT DESIGN PHASE	13
6	PROJECT PLANNING AND SCHEDULING	15
7	FUNCTIONAL AND PERFORMANCE TESTING	75
8	ADVANTAGES AND DISADVANTAGES	76
9	CONCLUSION	76

INTRODUCTION

1.1 Project Overview :

WhatNext Vision Motors implemented a Salesforce-based CRM solution to modernize vehicle ordering and dealership operations. The system automates vehicle order placement, validates stock availability, assigns orders to the nearest dealer, schedules test drive reminders, and updates order statuses dynamically. This transformation replaces manual and disconnected processes with a centralized, automated platform, improving operational efficiency and customer satisfaction.

WhatNext Vision Motors is a vehicle management application designed toorganize vehicle-related business activities. Tinformation, and handling vehicle orders efficiently. Salesforce is used as the development platform to support data management and business process automation. The application aims to reduce manual work, improve data accuracy, and provide easy access to vehicle information. By using Salesforce features such as Apex classes, triggers, trigger handlers, flows, and Batch Apex, the system can automate repetitive tasks and manage records effectively. Overall, WhatNext Vision Motors aims to provide a reliable, organized, and efficient solution for vehicle management.

1.2 Purpose :

The purpose of this project is to automate vehicle ordering, stock management, dealer assignment, and customer engagement using Salesforce CRM. The system aims to reduce manual intervention, eliminate order errors, provide real-time tracking, and enhance overall business productivity while delivering a seamless customer experience.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	**Iam (Customer)**	**I'm trying to**	**But**	**Because**	**Which makes me feel**
PS-1	Operations Manager	manage vehicle orders	stock info is manual	systems are disconnected	stressed
PS-2	Customer	place vehicle orders	availability unclear	delayed updates	uncertain

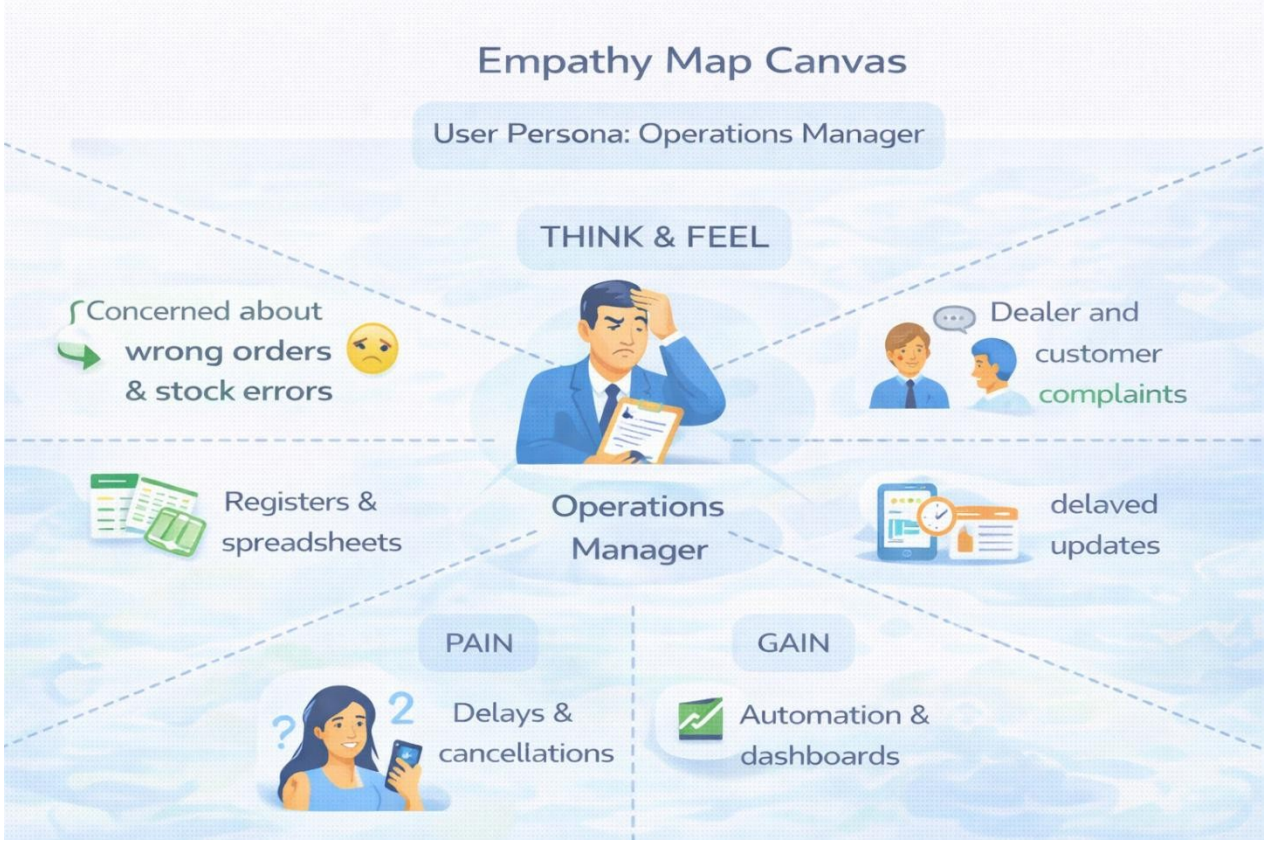
The system aims to reduce manual intervention, eliminate order errors, provide real-time tracking, and enhance overall business productivity while delivering a seamless customer experience.

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	 Operations Manager	manage vehicle orders	stock info is manual	 systems are disconnected	 stressed
PS-2	 Customer	place vehicle orders	availability unclear	 delayed updates	 uncertain

2.2 Empathy Map Canvas

User Persona: Operations Manager

- **Think & Feel:** Concerned about wrong orders & stock errors
- **Hear:** Dealer and customer complaints
- **See:** Registers & spreadsheets
- **Say & Do:** Manual follow-ups
- **Pain:** Delays & cancellations
- **Gain:** Automation & dashboards



2.3 Brainstorming & Idea Prioritization

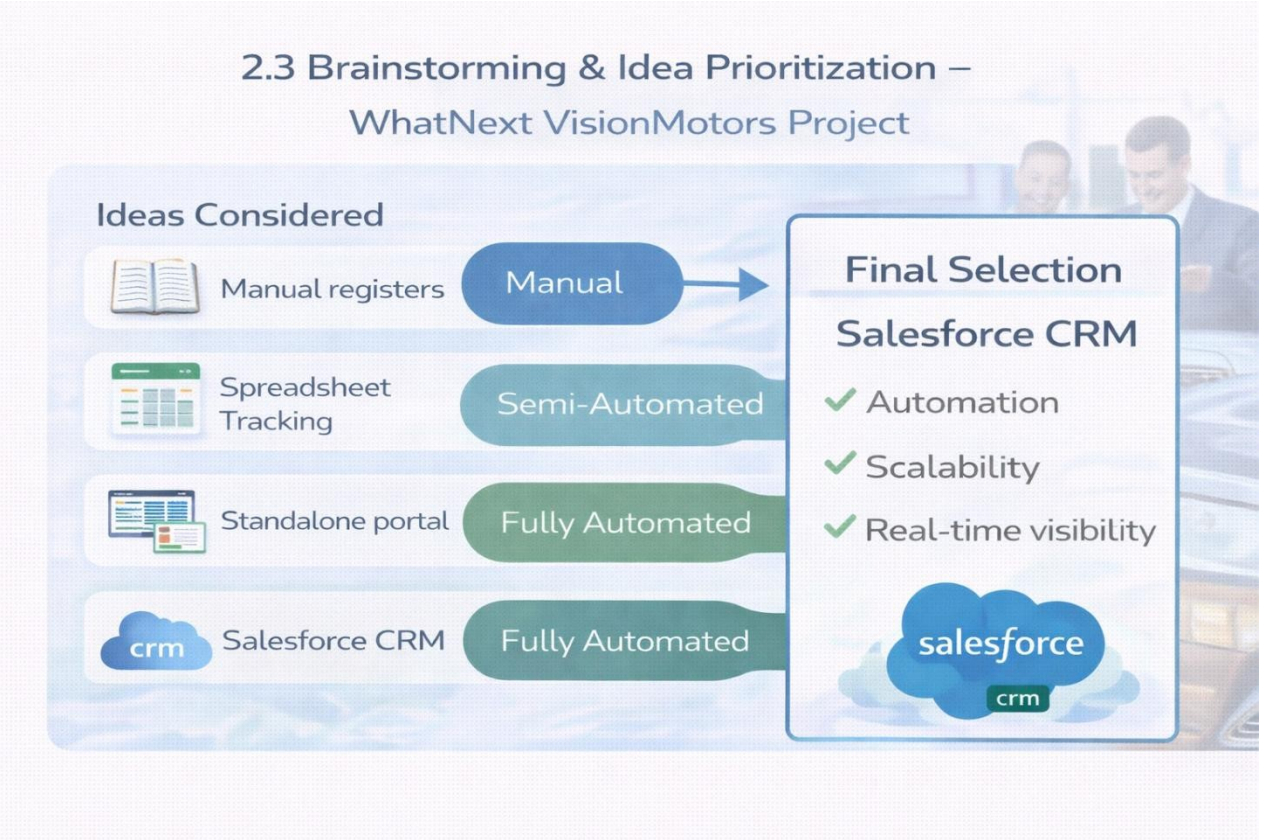
Ideas Considered

- Manual registers
- Spreadsheet tracking
- Standalone portal
- Salesforce CRM

Final Selection

Salesforce CRM selected due to:

- Automation
- Scalability



3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

- Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting

```
print("WhatNext Vision Motors")

print("Customer Journey Map")



journey = [
```

```
        "1. Customer Visits Website",
        "2. Browses Available Vehicles",
        "3. Selects a Vehicle",
        "4. Enters Customer Details",
        "5. Places an Order",
        "6. Receives Order Confirmation"
    ]

    for step in journey:
        print(step)

    print("Customer Journey Completed!")
```

```
public class VehicleManagement {

public static void displayDetails() {
    System.debug('WhatNext Vision Motors');
    System.debug('Vehicle Management System');
    System.debug('Vehicle Name: Tata Nexon');
    System.debug('Vehicle Status: Available');}}
```



Solution Requirements

3.2 Functional Requirements

FR No	Requirement
FR-1	Vehicle management
FR-2	Dealer management
FR-3	Customer order creation
FR-4	Stock validation
FR-5	Dealer auto-assignment
FR-6	Test drive scheduling
FR-7	Reports & dashboards

Non-Functional Requirements

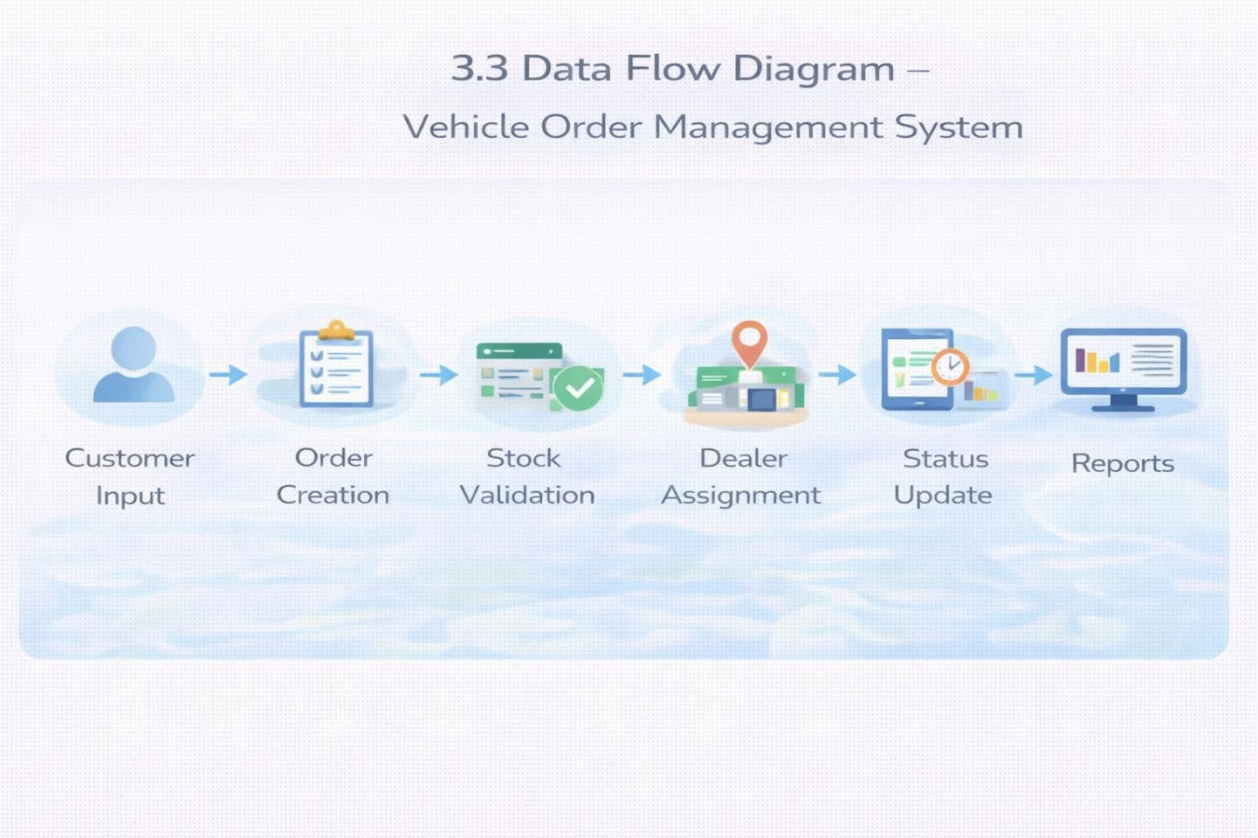
NFR	Description
Usability	Lightning UI for easy navigation
Security	Role-based access control
Reliability	Accurate automation
Performance	Fast order processing
Availability	24/7 cloud access
Scalability	Multi-dealer support

3.3 Data Flow Diagram

Flow:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports

Data flow describes how information moves through the WhatNext Vision Motors application. Users enter vehicle details, customer information, and order details into the system. Salesforce processes and stores the information in the database. Apex classes, triggers, and flows help validate data and automate required operations. The processed information can then be retrieved and displayed to users whenever needed. This data flow helps maintain data accuracy, improve efficiency, and support smooth vehicle management.

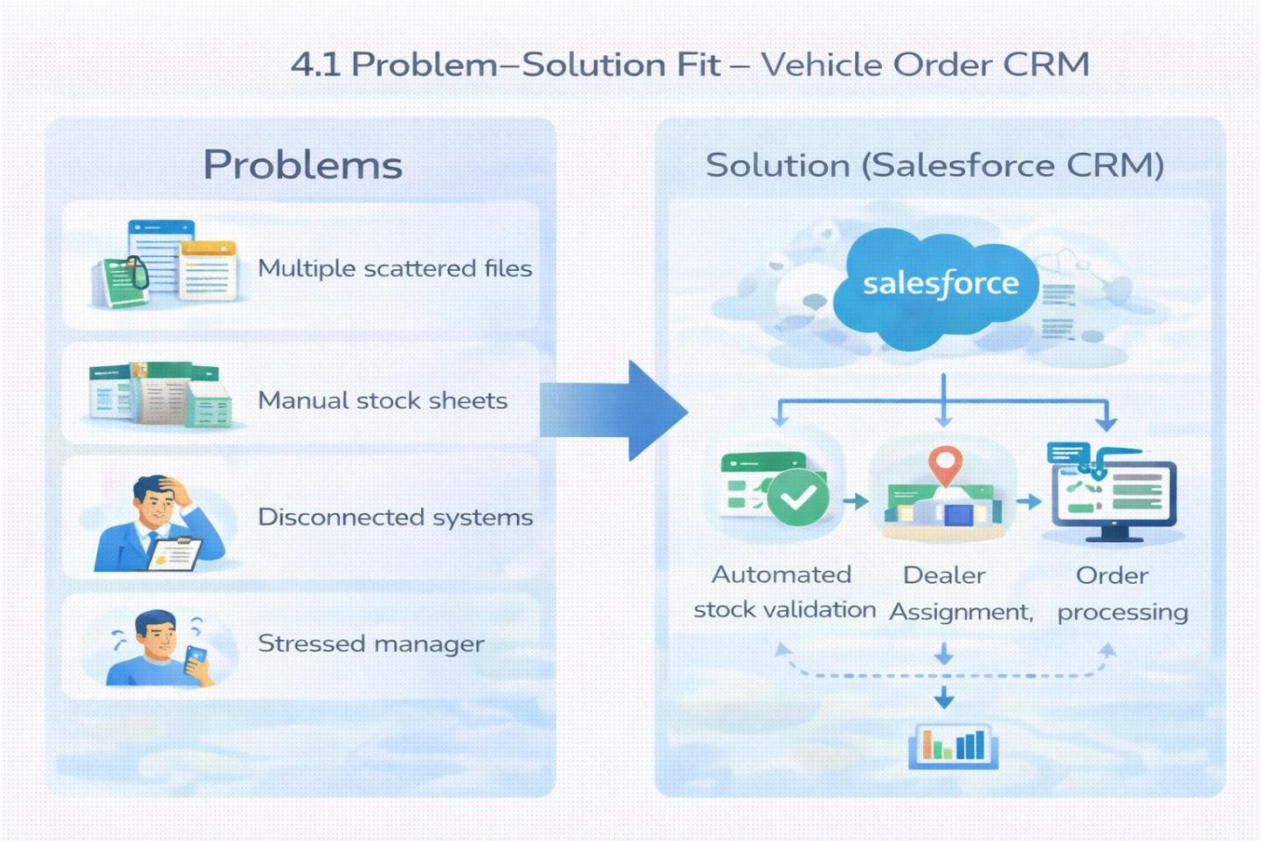


3.4 Technology Stack

Layer	**Technology**
UI	Salesforce Lightning
Logic	Salesforce Flows
Database	Salesforce Objects
Automation	Validation Rules, Triggers
Processing	Batch Apex, Scheduled Apex
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit



The system eliminates fragmented operations by integrating automated stock validation, dealer assignment, and order processing within Salesforce.

Problem	Manual vehicle ordering
Solution	Salesforce CRM
Innovation	Automated dealer assignment
Impact	Faster service
Business Model	Dealer network
Scalability	Multi-location support

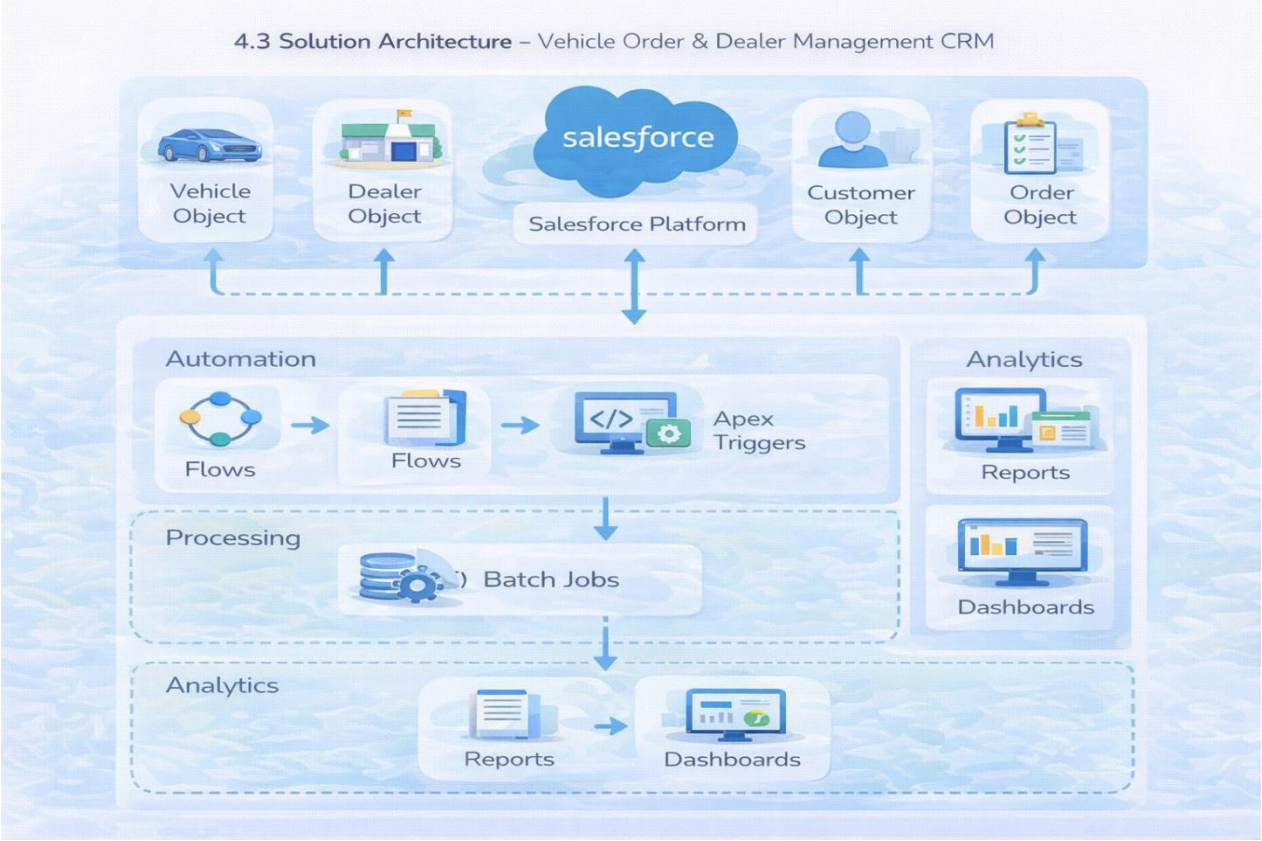
4.2 Solution Architecture

Components:

- Vehicle Object
- Dealer Object
- Customer Object
- Order Object
- Test Drive Object
- Service Request Object
- Flows
- Apex Triggers
- Batch Jobs
- Reports
- Dashboards

The solution architecture of WhatNext Vision Motors describes the overall structure and components of the application. Salesforce is used as the main development platform to manage vehicle details, inventory, customer information, and orders. The user interface allows users to interact with the application and access the required information. Salesforce objects store the data, while Apex classes, triggers, trigger handlers, and flows support business logic and process automation. Batch Apex can be used to process large volumes of records efficiently. This architecture helps improve data management, maintain accuracy, and provide a reliable and scalable vehicle.

5. PROJECT PLANNING & SCHEDULING



5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation

1. Product Backlog

The product backlog contains the list of features and tasks required to develop the WhatNext Vision Motors application.

Sprint 1 – Developer Setup

User Story (USN-1): As a system administrator, I want to create and configure a Salesforce

developer account so that I can develop and manage the WhatNext Vision Motors application.

Story Points: 3

Priority: High

Sprint 2 – Vehicle Management

User Story (USN-2): As an administrator, I want to add, update, and manage vehicle details so that vehicle information is maintained accurately.

Story Points: 5

Priority: High

Sprint 3 – Inventory Management

User Story (USN-3): As an administrator, I want to track vehicle availability and inventory so that available vehicles can be managed efficiently.

Story Points: 5

Priority: High

Sprint 4 – Customer Management

User Story (USN-4): As a user, I want to register and manage customer details so that customer information can be maintained efficiently.

Story Points: 5

Priority: Medium

Sprint 5 – Order Management

User Story (USN-5): As a customer, I want to place and track vehicle orders so that I can manage my vehicle purchases easily.

Story Points: 8

Priority: High

Sprint 6 – Testing and Deployment

User Story (USN-6): As an administrator, I want to test and deploy the application so that the system functions correctly and is ready for use.

Story Points: 5

Priority: High

3. Sprint Schedule

The project is divided into six sprints. Each sprint focuses on a specific functional requirement, beginning with developer setup and progressing through vehicle management, inventory management, customer management, order management, and final deployment.

4. Estimation

Story points are used to estimate the complexity and effort required to complete each user story. Higher story points indicate tasks that require relatively more effort.

The total estimated effort for the six sprints is 31 story points. The estimation may be adjusted based on development progress, testing results, and changes in project requirements.

5. Expected Outcome

The WhatNext Vision Motors application aims to provide an organized system for managing vehicle details, inventory, customer information, and vehicle orders. Agile sprint planning helps ensure structured development, continuous testing, and reliable application delivery.

6. Project Planning and Scheduling

Project planning and scheduling help organize the development activities of the WhatNext Vision Motors application. The project is divided into different phases to ensure that each task is completed in a systematic manner.

Project Planning

The project planning phase identifies the objectives, requirements, resources, and activities needed to develop the application. The main goal is to create an efficient vehicle management system that simplifies vehicle information management, inventory tracking, customer management, and order processing.

The major planning activities include:

Identifying project requirements and objectives.

Defining the application features and functionalities.

Allocating tasks based on project priorities.

Estimating the time and effort required for each activity.

Planning testing and deployment activities.

Project Schedule

The project schedule organizes the development activities into the following phases:

Phase 1: Requirement Analysis

Collecting and analyzing the requirements of the WhatNext Vision Motors application.

Phase 2: System Design

Designing the application structure, data model, user interface, and required functionalities.

Phase 3: Development

Implementing vehicle management, inventory management, customer management, and order management features.

Phase 4: Testing

Testing the application to identify errors and ensure that the implemented features work correctly.

Phase 5: Deployment

Deploying the completed application and verifying its functionality in the target environment.

Project Monitoring

Project progress is monitored throughout the development process. Each sprint is reviewed to identify completed tasks, pending activities, and issues that may affect the schedule. Necessary changes are made to ensure that the project meets its objectives.

Risk Management

Risk management helps identify potential problems that may affect the development of the WhatNext Vision Motors application.development. These risks can be reduced through proper planning, regular testing, clear documentation, and project progress.

Project Development Phase:

What you'll learn

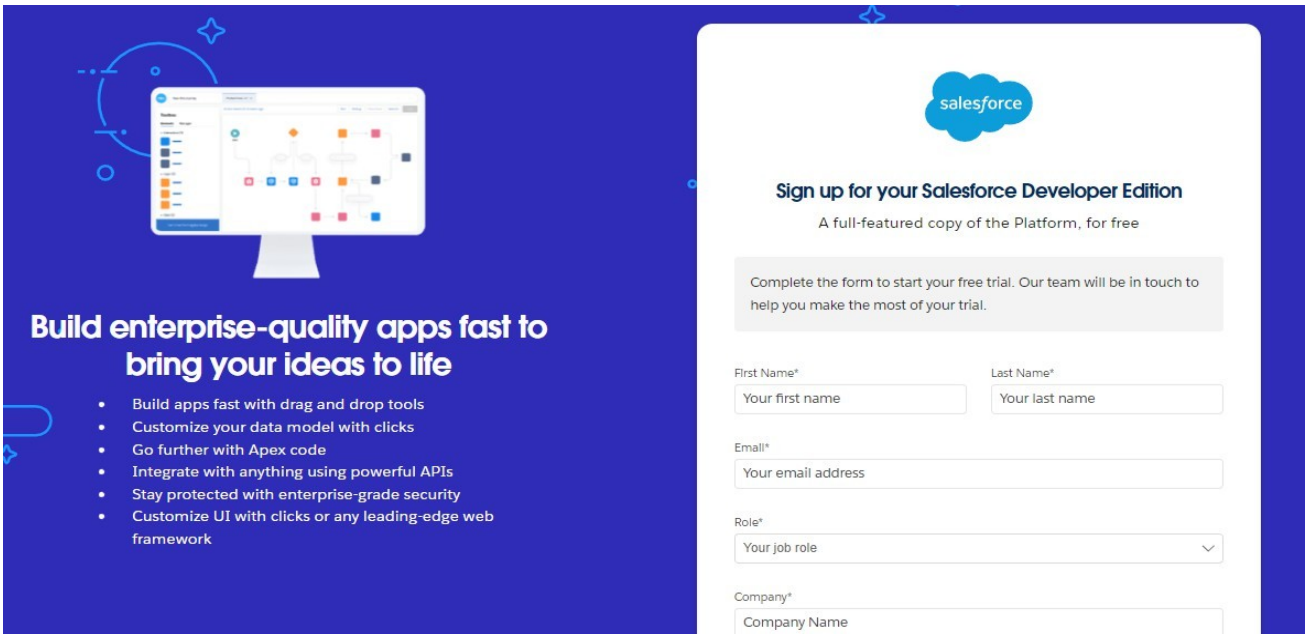
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

Milestone 1: Salesforce Account

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to https://developer.salesforce.com/signup
2. On the sign up form, enter the following details :

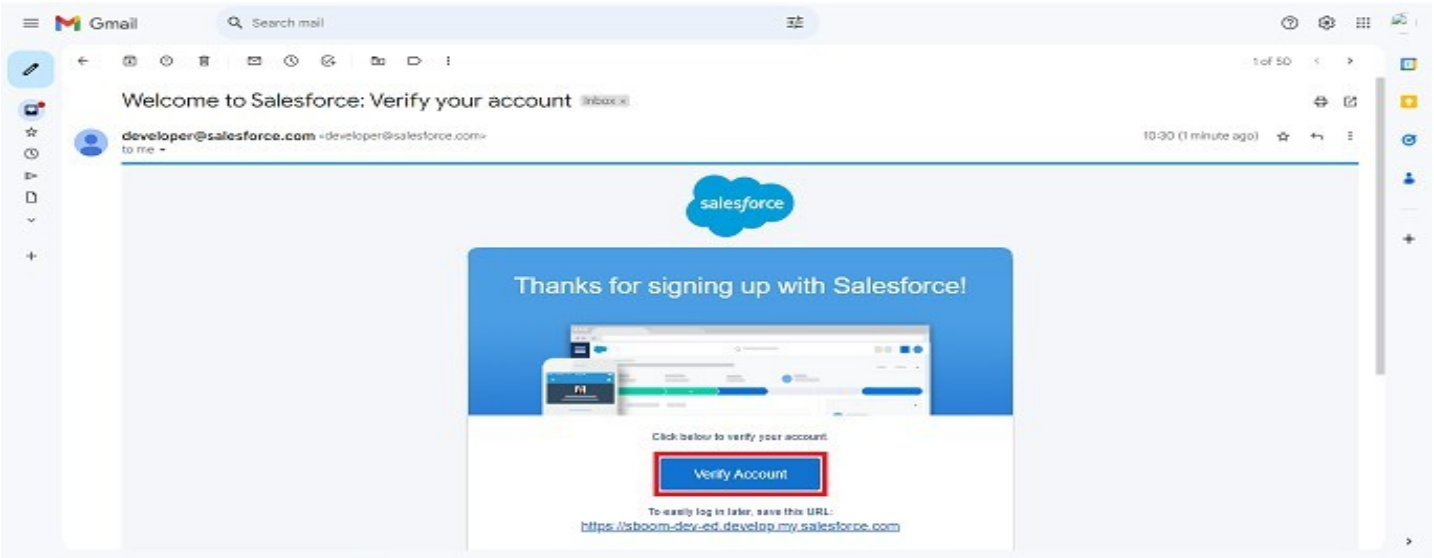


1) First name & Last name
2) Email
3) Role : Developer
4) Company : College Name
5) County : India
6) Postal Code : pin code
7) Username : should be a combination of your name and company This need not be an actual email id, you can give anything in the format : username@organization.com

Click on sign me up after filling these.

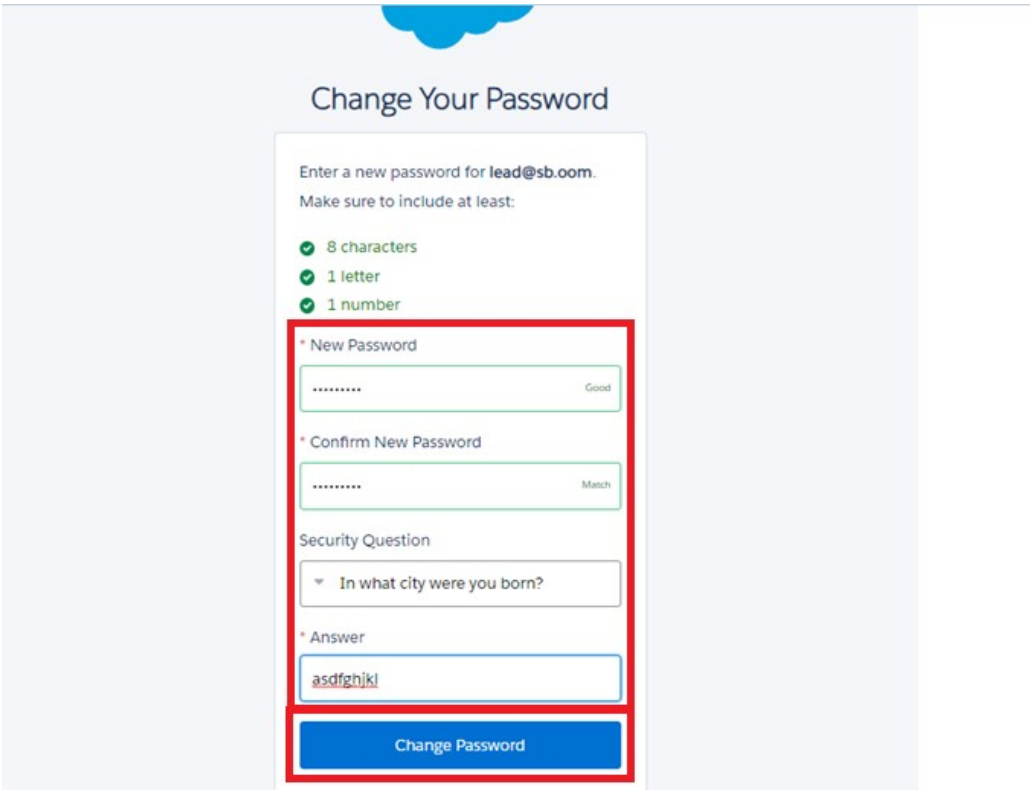
Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.

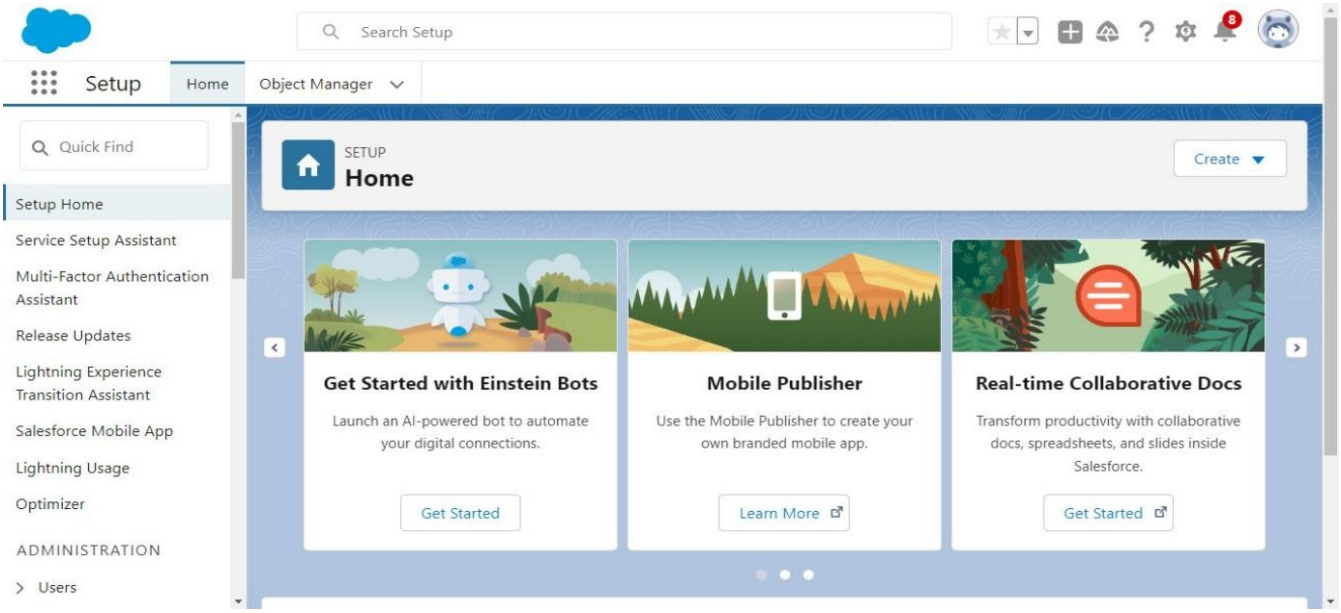


2.Click on Verify Account

3.Give a password and answer a security question and click on change password.



2. Then you will redirect to your salesforce setup page.



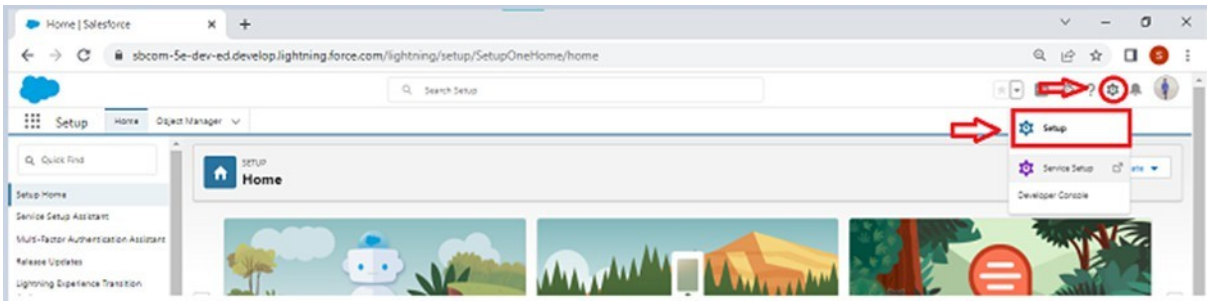
Milestone 2: Data Management-Objects

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Reques t_c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.



Activity 1: Create Vehicle Object

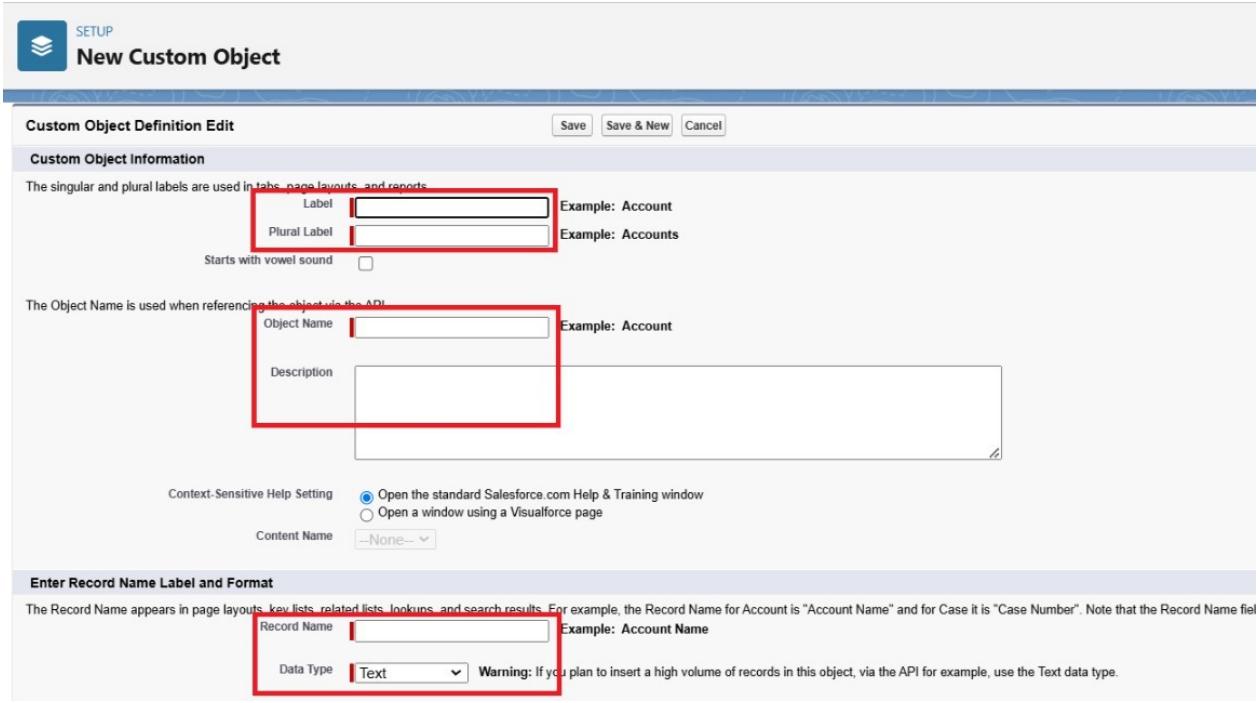
The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

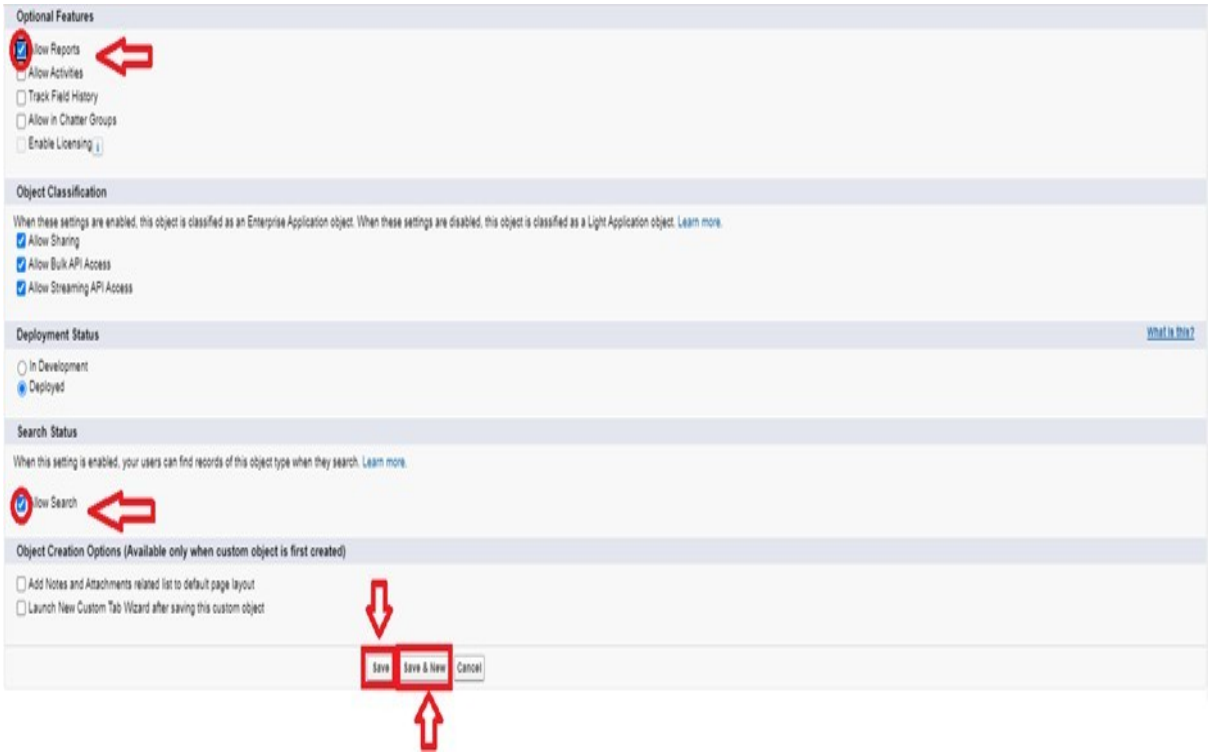
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



1) Enter the label name → Vehicle
2) Plural label name → Vehicles
3) Enter Record Name Label and Format
   - Record Name → Vehicle Name
   - Data Type → Text



2. Click on Allow reports,
3. Allow search → **Save.**



Activity 2: Create Dealer Object

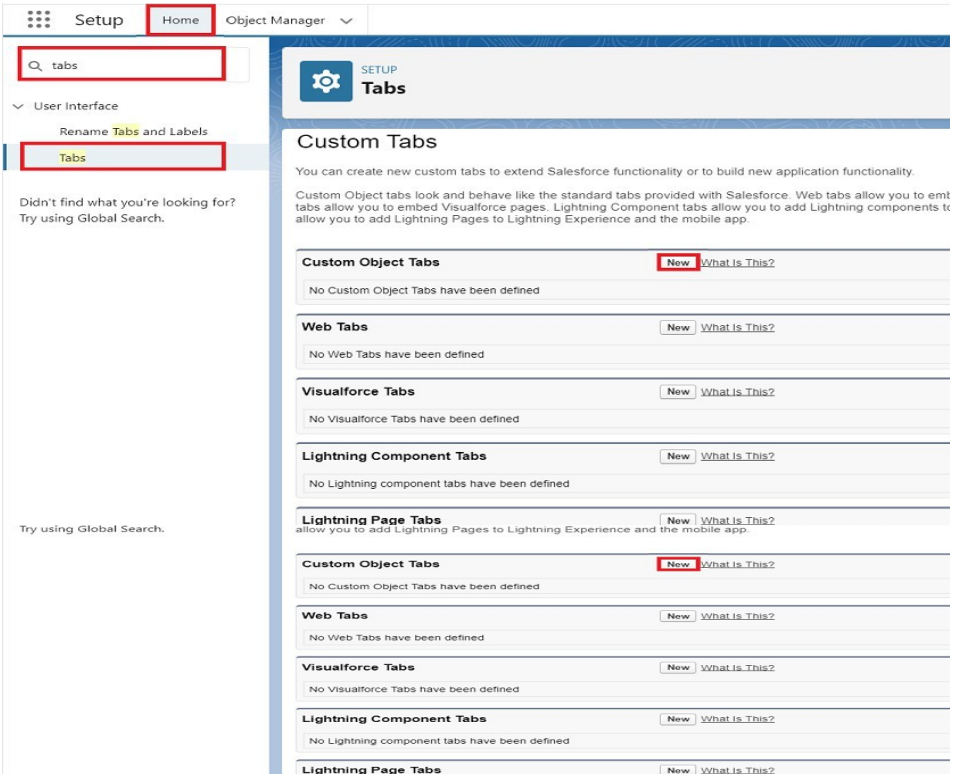
The purpose of creating a Dealer object is to have detailed information about Dealer. To create an object:

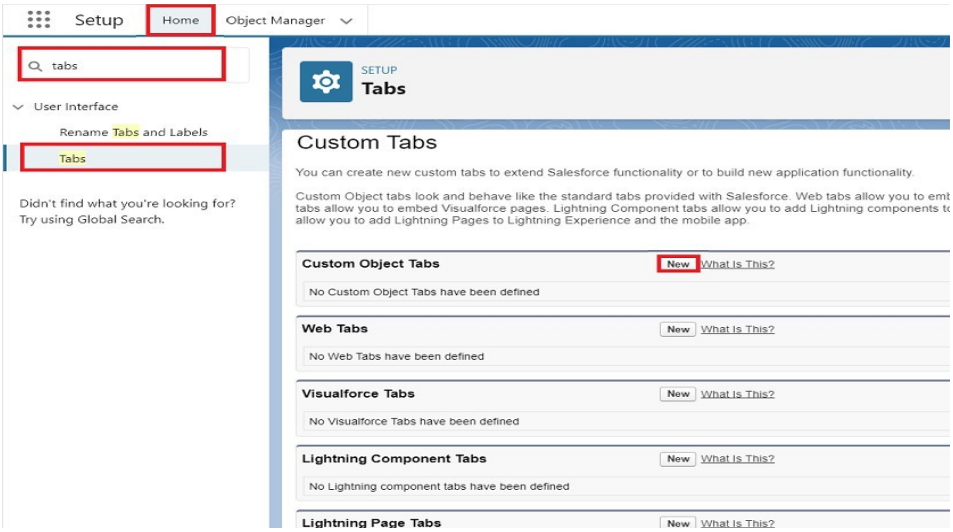
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    1) Enter the label name → Vehicle Dealer
    2) Plural label name → Vehicle Dealers
    3) Enter Record Name Label and Format
        - Record Name → Dealer Name
        - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save as**

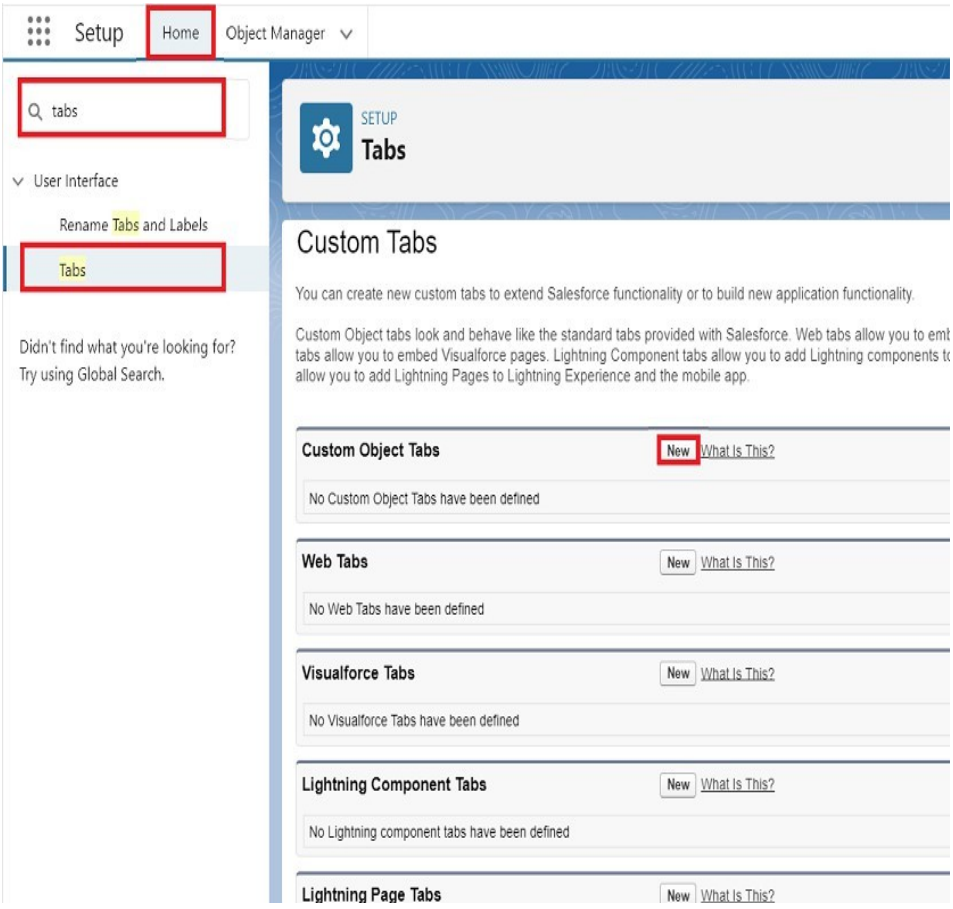
Activity 3: Creating remaining objects

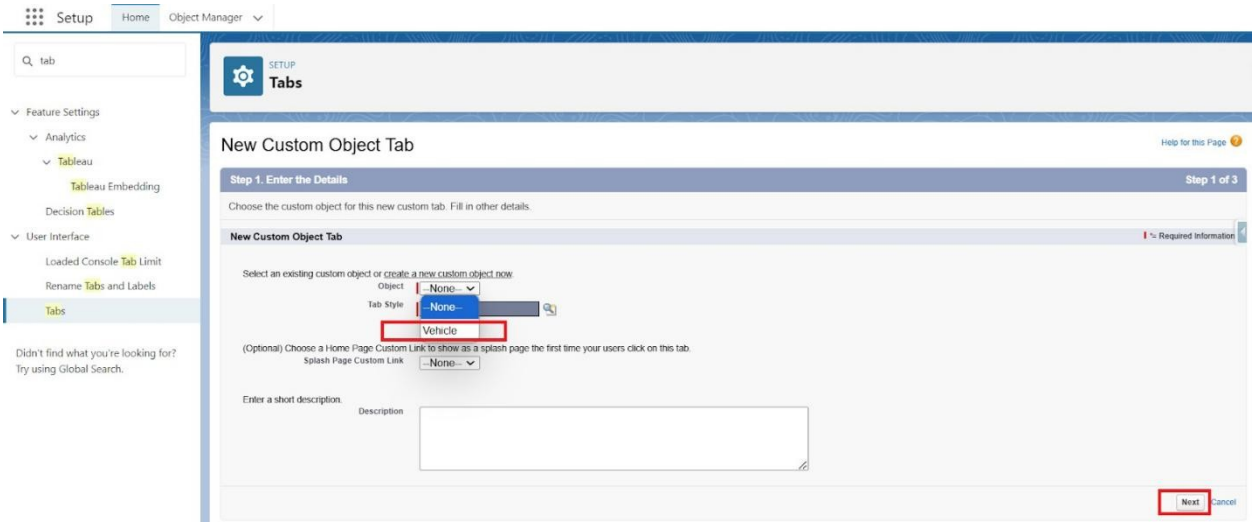
Milestone 3: Data Management-Tabs

Activity 1: Creating a Custom Tab(Vehicle)




Setup
Home
Object Manager
tabs
User Interface
Rename Tabs and Labels
Tabs
Didn't find what you're looking for?
Try using Global Search.
SETUP
Tabs
Custom Tabs
You can create new custom tabs to extend Salesforce functionality or to build new application functionality.
Custom Object tabs look and behave like the standard tabs provided with Salesforce. Web tabs allow you to emb
tabs allow you to embed Visualforce pages. Lightning Component tabs allow you to add Lightning components to
allow you to add Lightning Pages to Lightning Experience and the mobile app.
Custom Object Tabs
New
What Is This?
No Custom Object Tabs have been defined
Web Tabs
New
What Is This?
No Web Tabs have been defined
Visualforce Tabs
New
What Is This?
No Visualforce Tabs have been defined
Lightning Component Tabs
New
What Is This?
No Lightning component tabs have been defined
Lightning Page Tabs
New
What Is This?


Setup
Home
Object Manager
tabs
User Interface
Rename Tabs and Labels
Tabs
Didn't find what you're looking for?
Try using Global Search.
SETUP
Tabs
Custom Tabs
You can create new custom tabs to extend Salesforce functionality or to build new application functionality.
Custom Object tabs look and behave like the standard tabs provided with Salesforce. Web tabs allow you to emb
tabs allow you to embed Visualforce pages. Lightning Component tabs allow you to add Lightning components to
allow you to add Lightning Pages to Lightning Experience and the mobile app.
Custom Object Tabs
New
What Is This?
No Custom Object Tabs have been defined
Web Tabs
New
What Is This?
No Web Tabs have been defined
Visualforce Tabs
New
What Is This?
No Visualforce Tabs have been defined
Lightning Component Tabs
New
What Is This?
No Lightning component tabs have been defined
Lightning Page Tabs
New
What Is This?



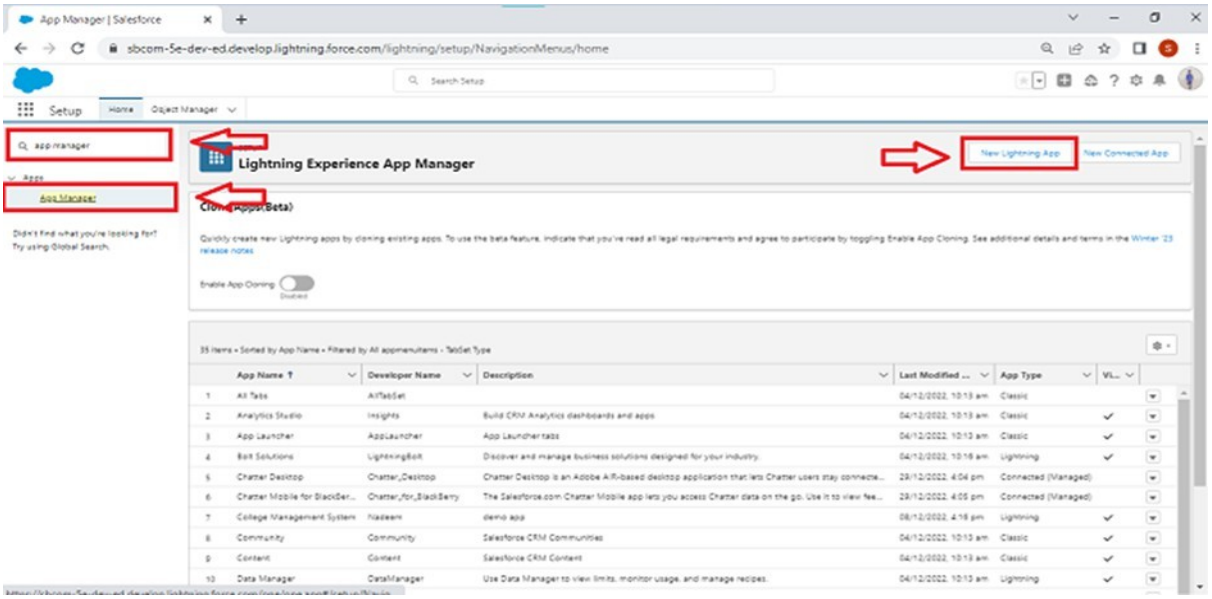
Activity 2:

Now create tabs for Other objects as well which we have created above Objects.

Milestone 4: Data Management-App Manager

Activity 1: Create a Lightning App To create a lightning app page:

1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



2. Fill the app name in app details and branding

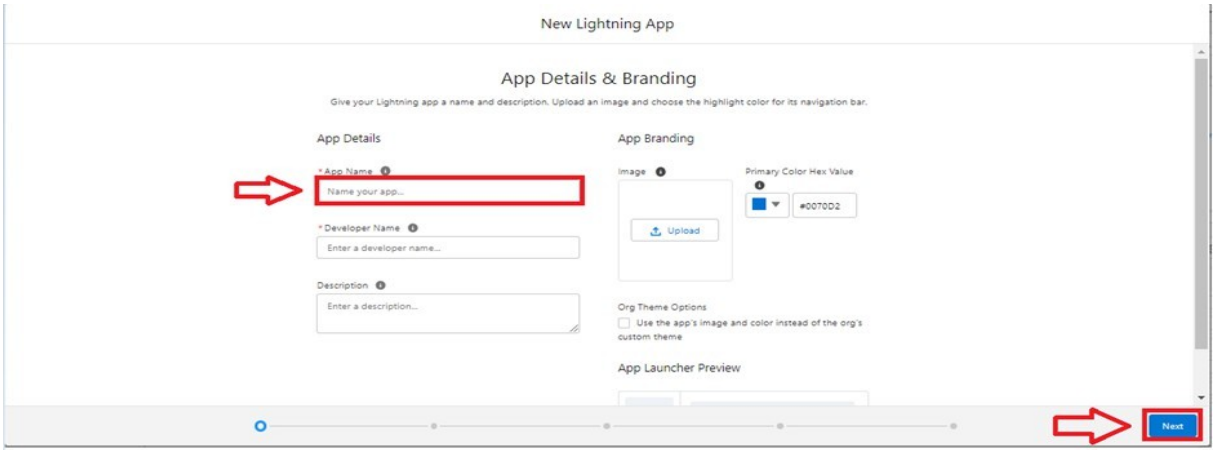
   as follow App Name : WhatNext Vision Motors

   Developer Name : this will auto populated

   Description : Give a meaningful description

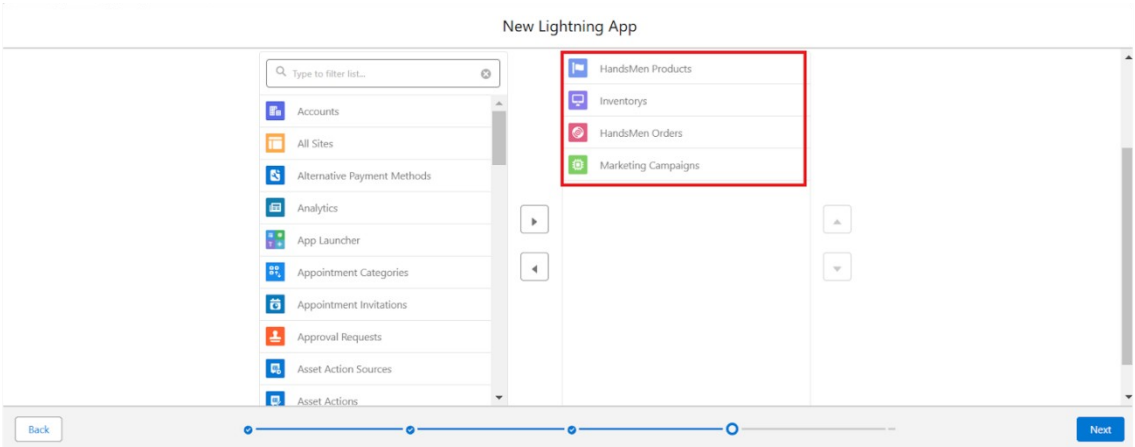
   Image : optional (if you want to give any image you can otherwise not mandatory) Primary color hex value : keep this default

3. Then click Next → (App option page) keep it as default → Next →



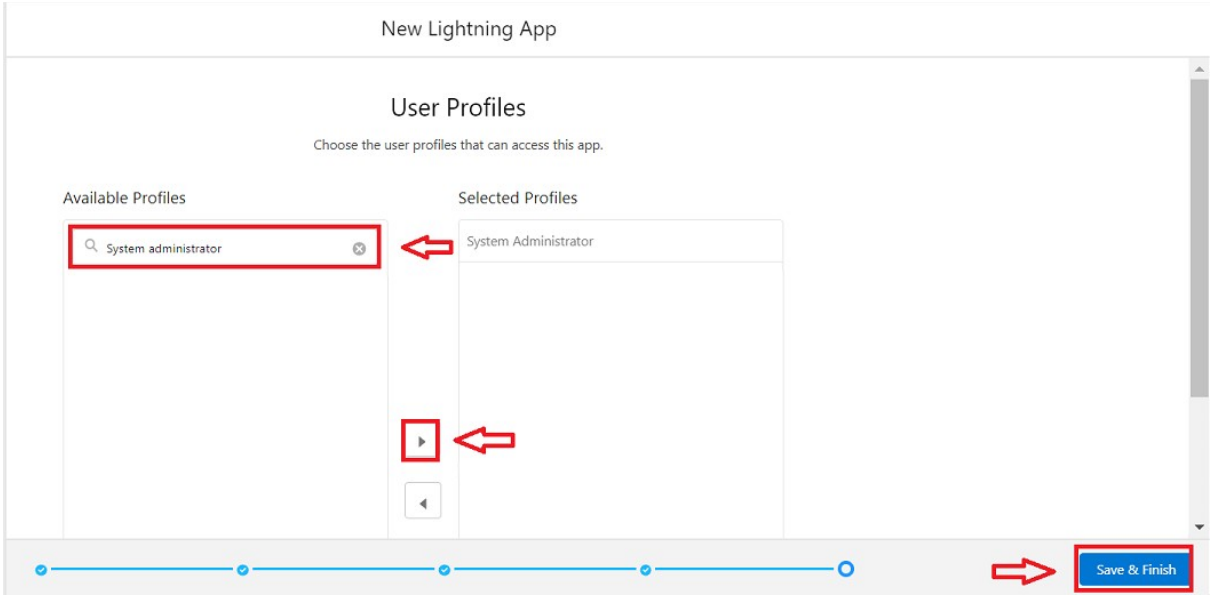
   (Utility Items) keep it as default → Next.

4. To Add Navigation Items:



   Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

   Note: select the custom object which we have created in the previous activity.



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5: Data Management-Fields

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle_Name__c** (Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_ Dealer__c (Custom Object)

- **Dealer_Name__c** (Text) **Vehicle_Order__c (Custom Object)**

Customer__c (Lookup to Customer__c)

- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

3. **Vehicle_Customer__c (Custom Object)**

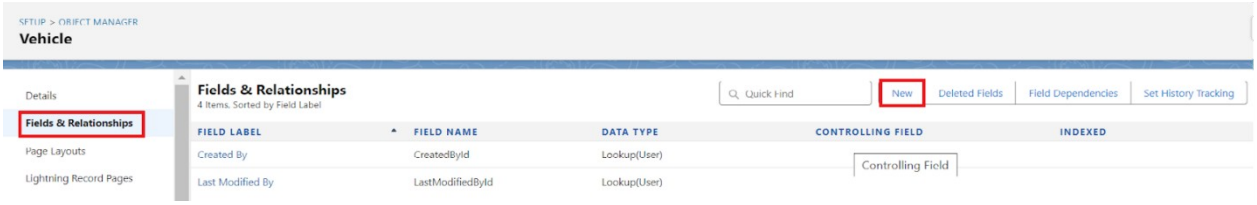
- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type_c** (Picklist: Sedan, SUV, EV, etc.)

4. **Vehicle_Test_Drive__c (Custom Object)**

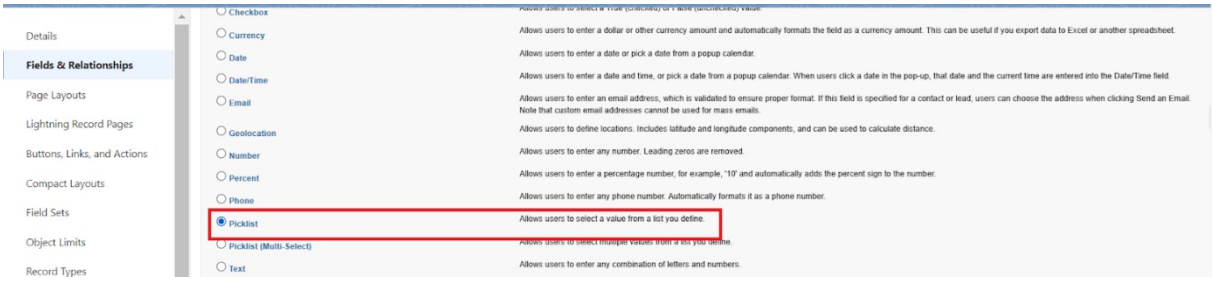
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

5. **Vehicle_Service_Request__c (Custom Object)**

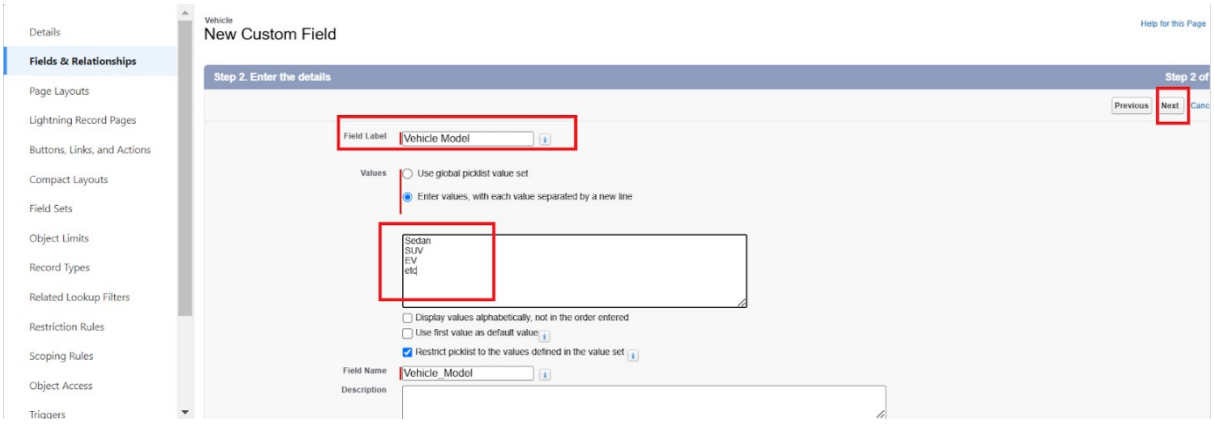
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)



1. Select Data type as “PickList”.



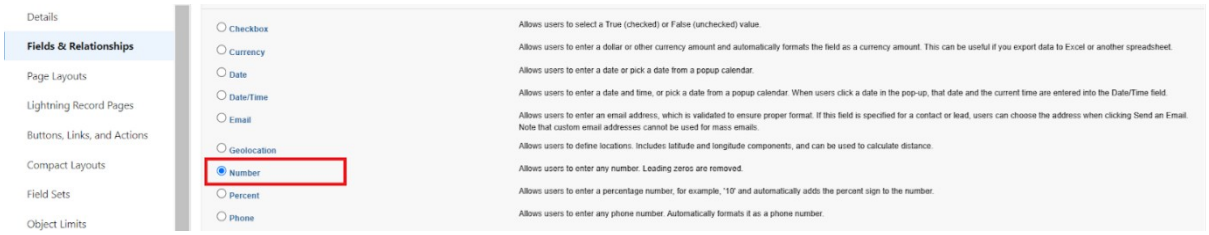
2. Click on Next



3. Fill the above as following:
   - Field Label: Vehicle Modal
   - Fill PickList Values
   - Click on Next → Next → Save and new.

Object Field Creation-2 :

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.




```
public class VehicleObjectController {

    // Create Vehicle Record
    public static void createVehicle() {

        Vehicle__c vehicle = new Vehicle__c();

        vehicle.Name = 'Tata Nexon';
        vehicle.Vehicle_Price__c = 850000;
        vehicle.Vehicle_Status__c = 'Available';

        insert vehicle;

        System.debug('Vehicle created successfully!');
    }

    // Display Vehicle Records
    public static void displayVehicles() {

        List<Vehicle__c> vehicles = [
            SELECT Id, Name, Vehicle_Price__c,
                   Vehicle_Status__c
            FROM Vehicle__c
        ];
```

```
    for (Vehicle__c v : vehicles) {
        System.debug('Vehicle Name: ' + v.Name);
        System.debug('Vehicle Price: ' + v.Vehicle_Price__c);
        System.debug('Vehicle Status: ' + v.Vehicle_Status__c);
    }
}

// Update Vehicle Details
public static void updateVehicle() {

    List<Vehicle__c> vehicles = [
        SELECT Id, Name, Vehicle_Price__c
        FROM Vehicle__c
        WHERE Name = 'Tata Nexon'
        LIMIT 1
    ];

    if (!vehicles.isEmpty()) {
        vehicles[0].Vehicle_Price__c = 900000;
        update vehicles[0];

        System.debug('Vehicle updated successfully!');
    }
}

// Delete Vehicle Record
public static void deleteVehicle() {

    List<Vehicle__c> vehicles = [
        SELECT Id, Name
        FROM Vehicle__c
```

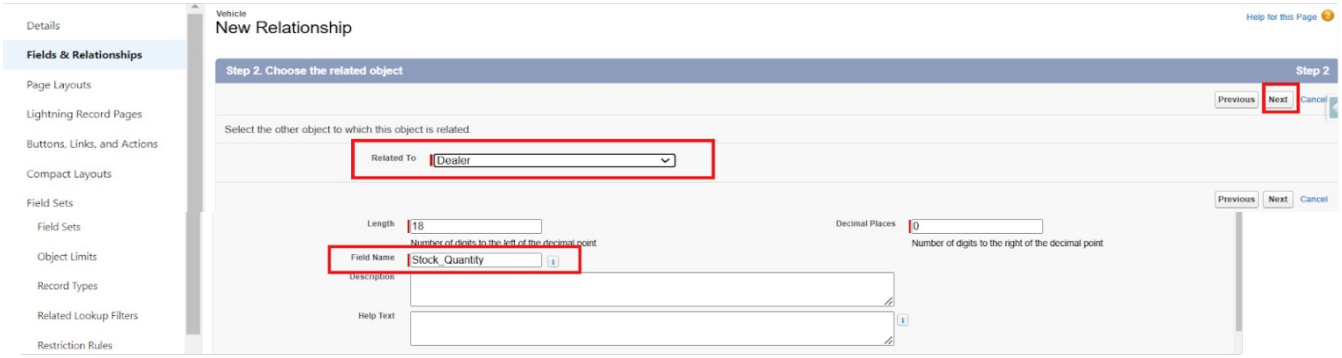
WHERE Name = 'Tata Nexon'






```
if (!vehicles.isEmpty()) {
        delete vehicles[0];

        System.debug('Vehicle deleted successfully!');
    }
  }
}
```

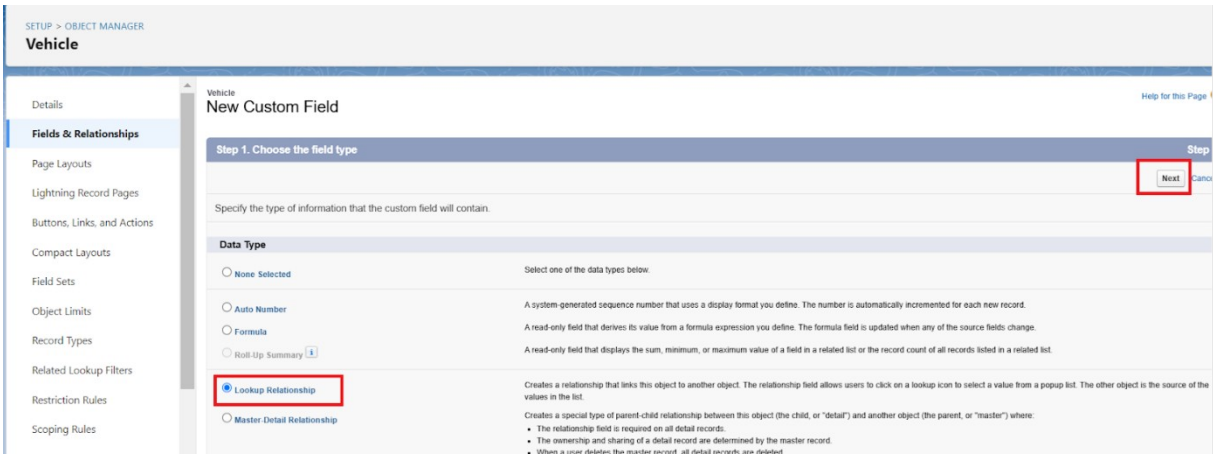


3. Click on Next

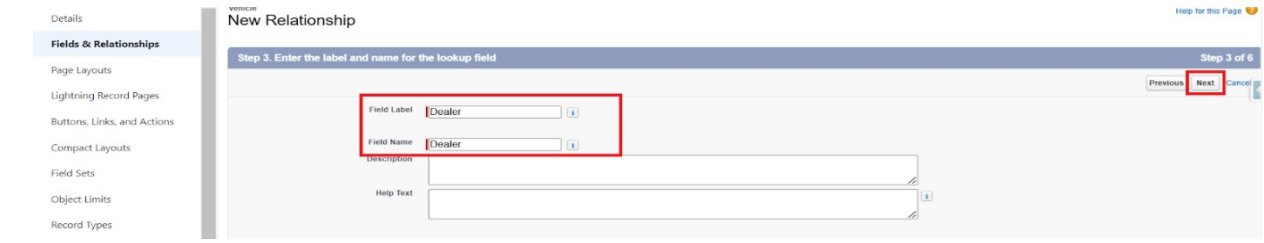
4. Fill the above as following:
   - ○ Field Label: Stock Quality
   - ○ Click on Next → Next → Save and new.

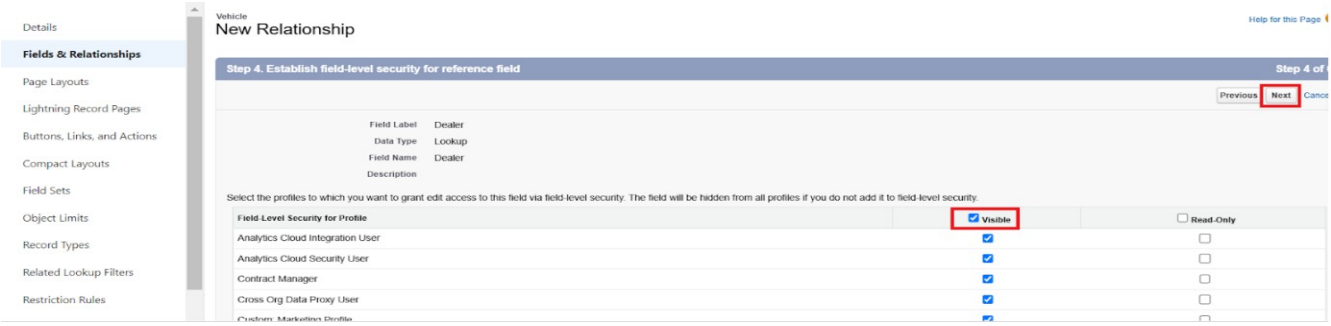
Relationship Field Creation :

Creating a Lookup RelationShip in Vehicle Object to Dealer Object.



1. Select DataType as Lookup Relationship.
2. Click Next
3. Select Dealer Object Form Related Filed





More Fields Creation

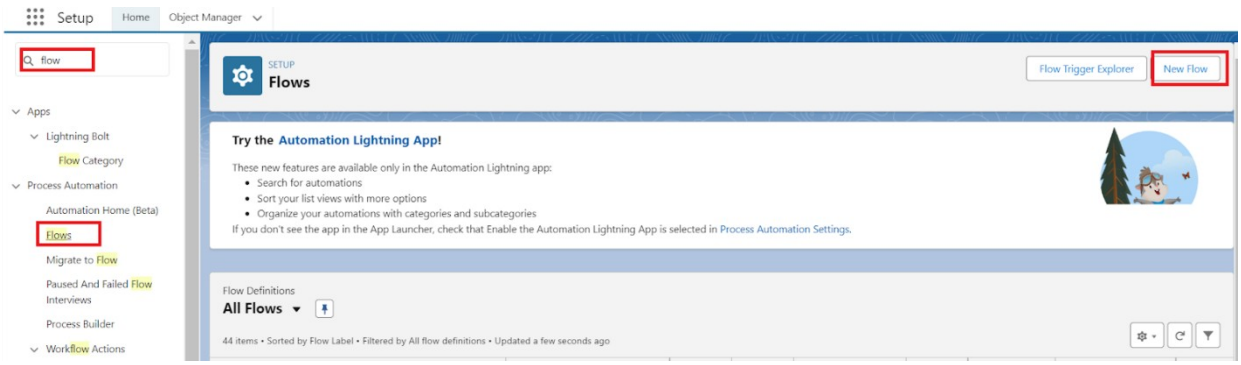
Create remaining fields as mentioned in the Data Management-Fields

Milestone 6: Automation

Flow Creation :

Creating a record triggered flow to assign nearest dealer to the customer's location

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Start From Scratch and click Next.

```
public class VehicleFlowController {

    public static void runVehicleFlow() {

        Map<String, Object> inputVariables =
            new Map<String, Object>();

        Flow.Interview.Vehicle_Management_Flow vehicleFlow =
            new Flow.Interview.Vehicle_Management_Flow(
                inputVariables
            );

        vehicleFlow.start();

        System.debug('WhatNext Vision Motors');
        System.debug('Vehicle Management Flow Started');
        System.debug('Flow executed successfully');

public class VehicleObjectController {

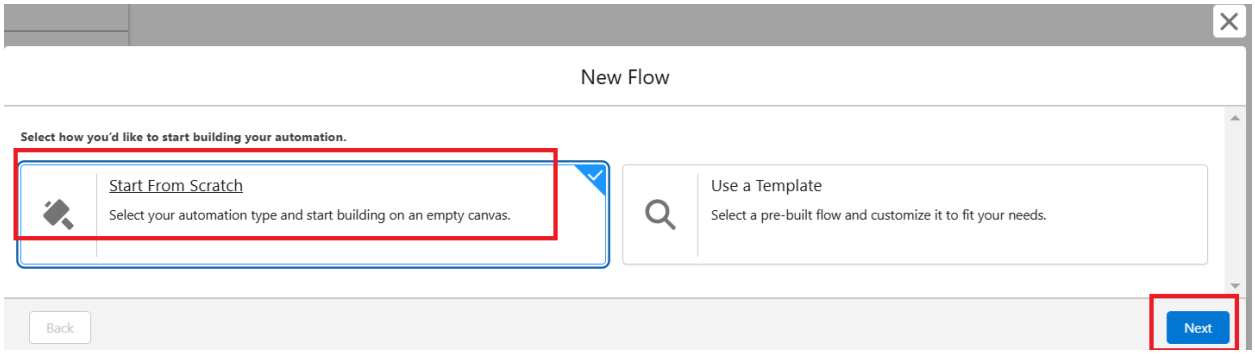
    // Create Vehicle Record
    public static void createVehicle() {

        Vehicle__c vehicle = new Vehicle__c();

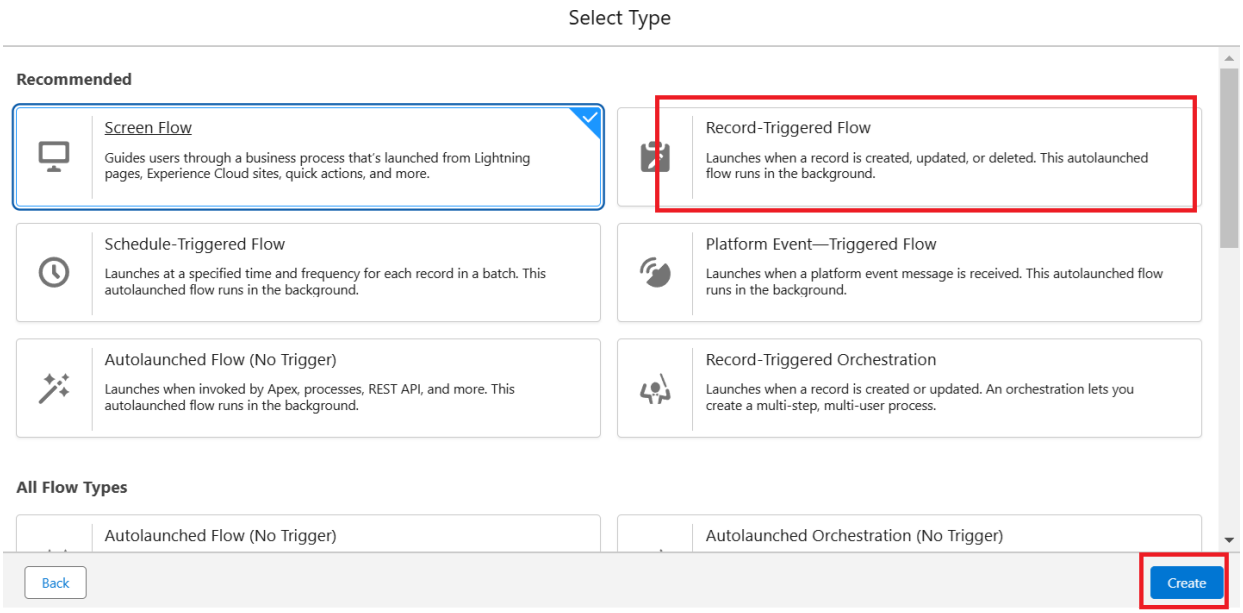
        vehicle.Name = 'Tata Nexon';
        vehicle.Vehicle_Price__c = 850000;
        vehicle.Vehicle_Status__c = 'Available';

        insert vehicle;
```

```
    System.debug('Vehicle created successfully!');
}
```



Step 3 : Select Record-Triggered Flow and click Create



Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met

(AND) Filed : Status__c

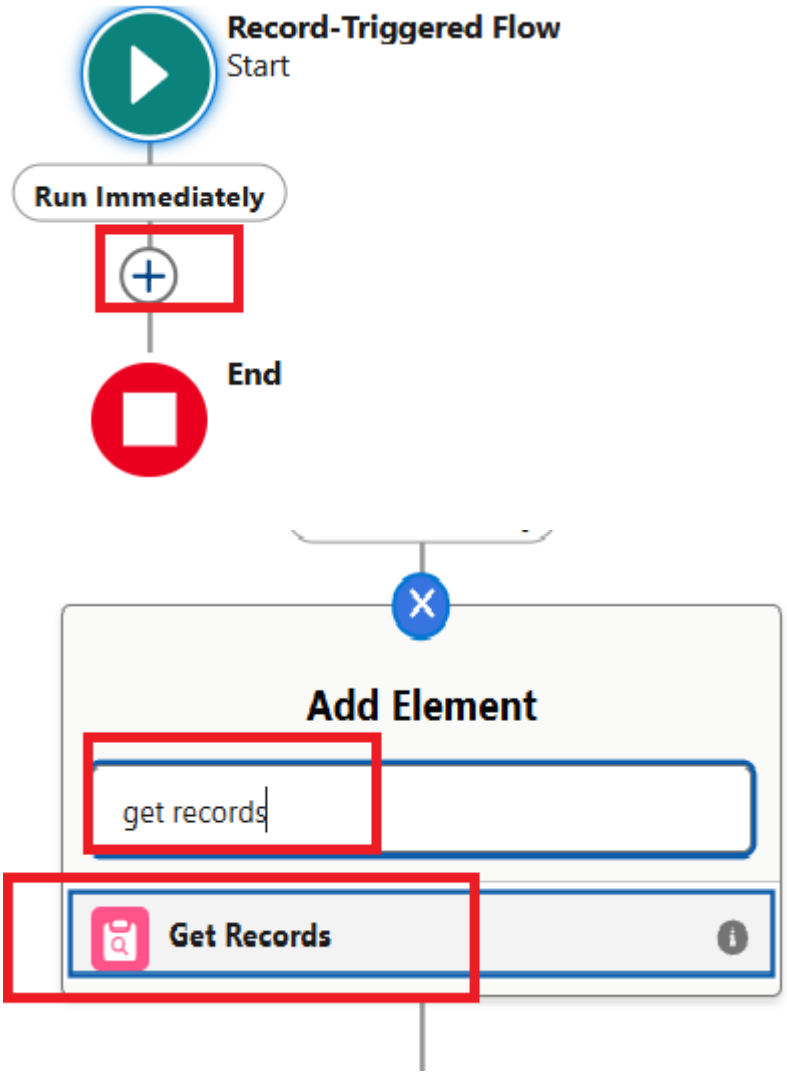
Operator :

Equals Value :

Pending

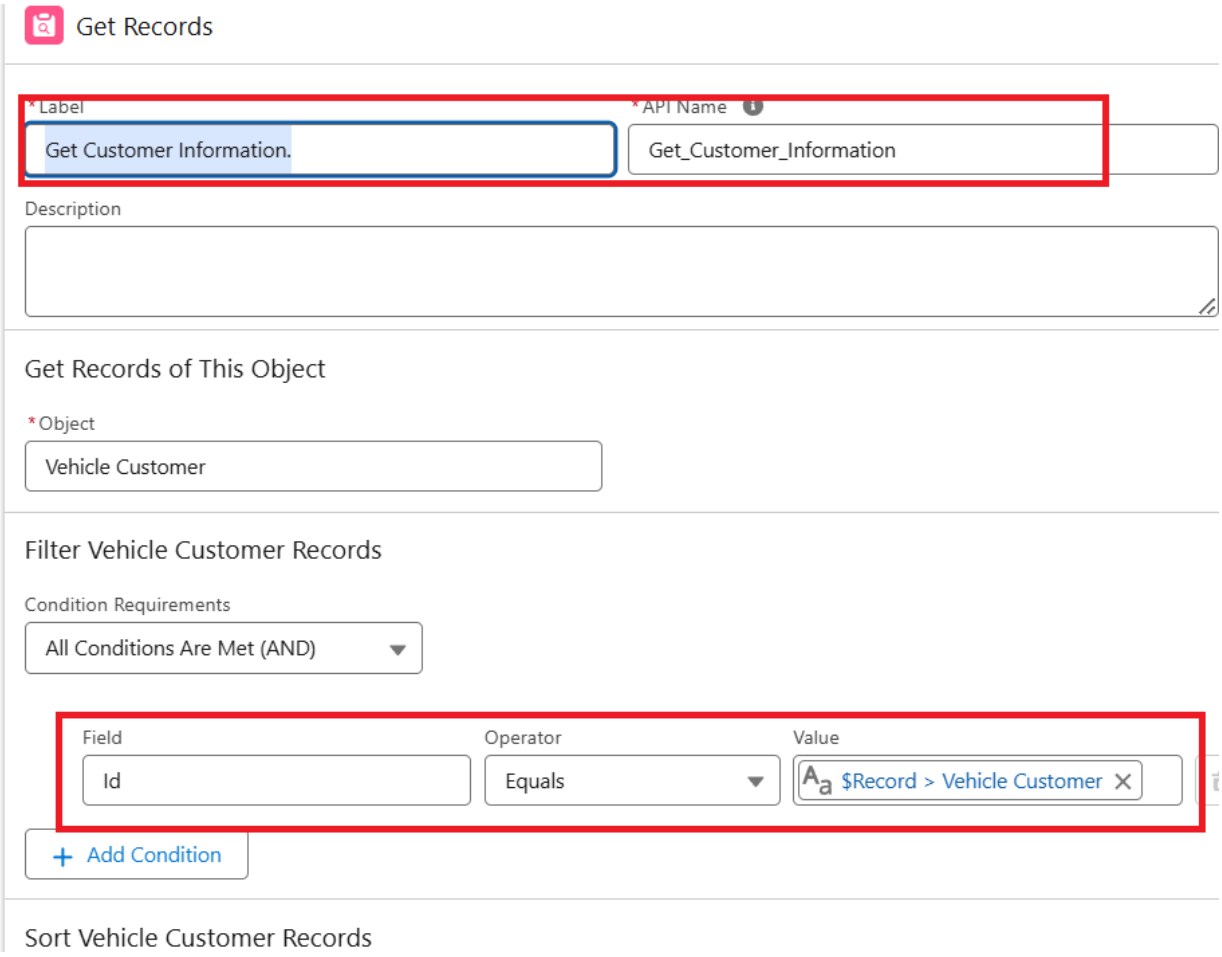
```
app = Flask(__name__)
```

Step 5 : Click + → Select **Get Records**



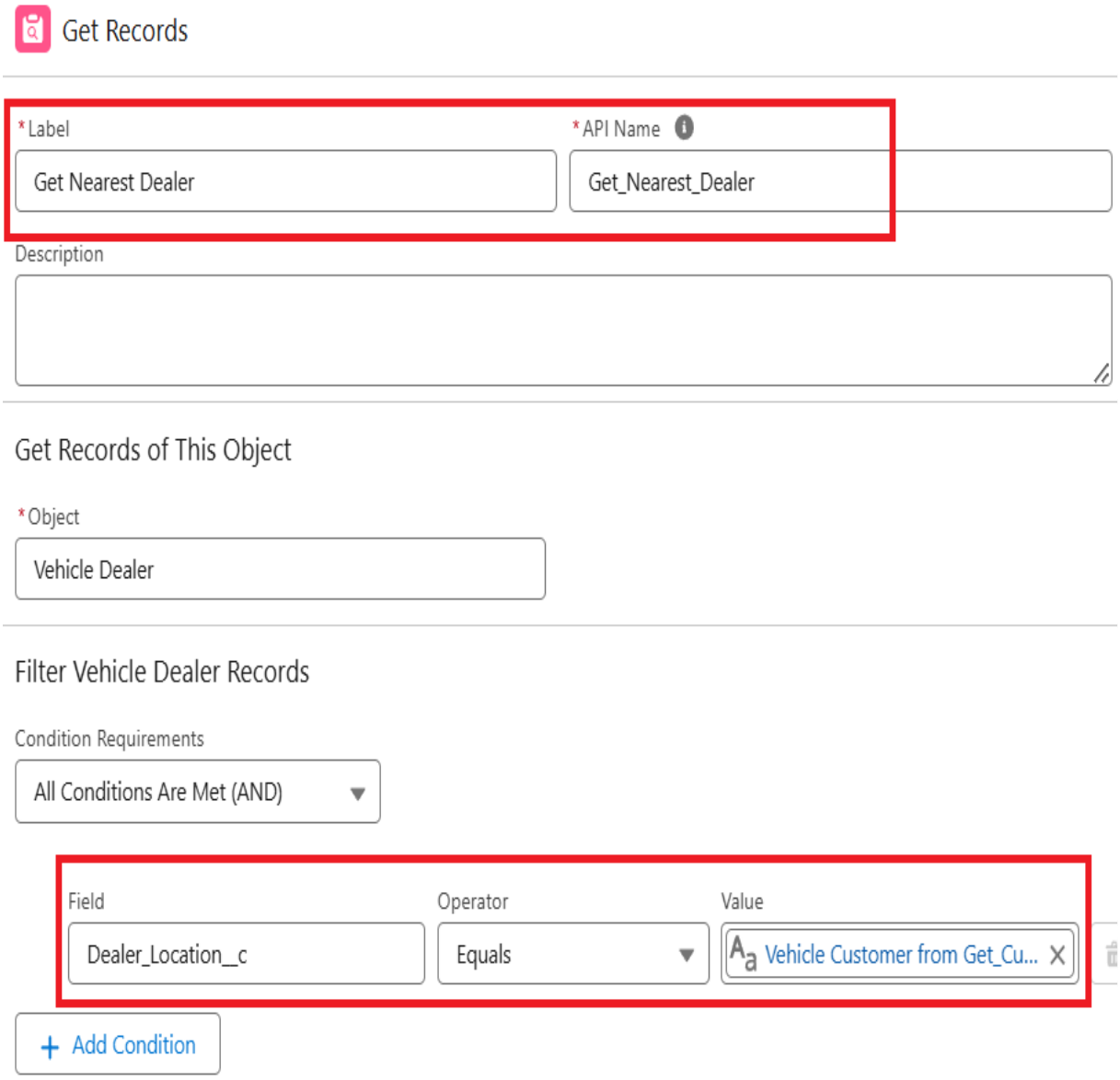
Step 6 : **Label :** Get Customer Information

- Object : Vehicle Customer
- Condition



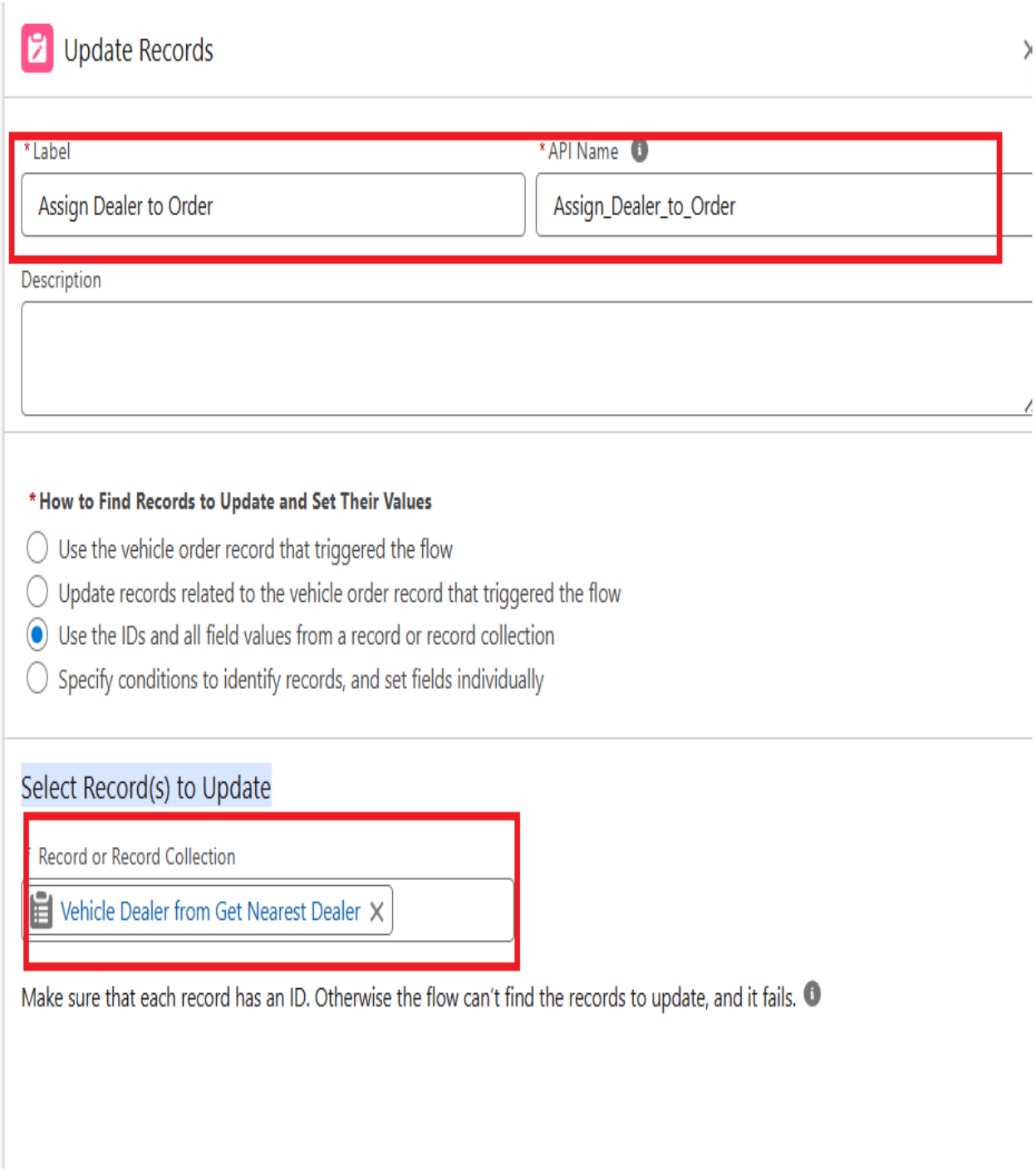
Step 7 : Click + → Select **Get Records**

- **Label :** Get Nearest Dealer
- Object : Vehicle Dealer
- Condition :
- Field : Dealer_Location__c
- Operator : Equals
- Value : {!Get_Customer_Information.Address__c}



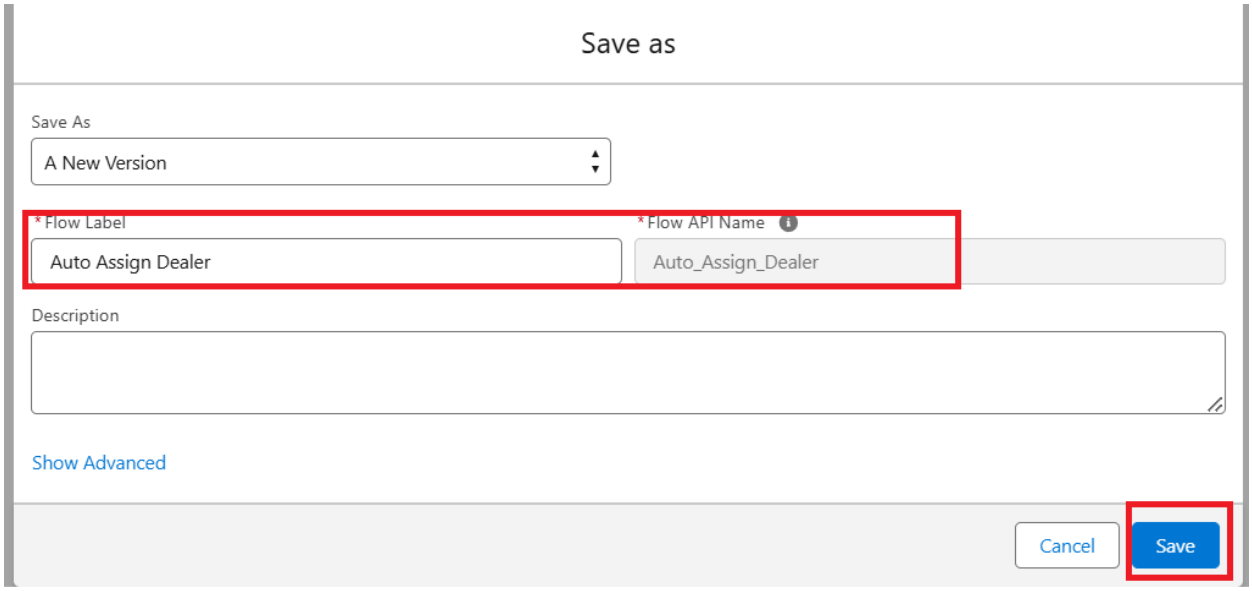
Step 8 : Click + → Select **Update Records**

- **Label :** Assign Dealer to Order
- *How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection
- Select Record(s) to Update : {!Get_Nearest_Dealer}

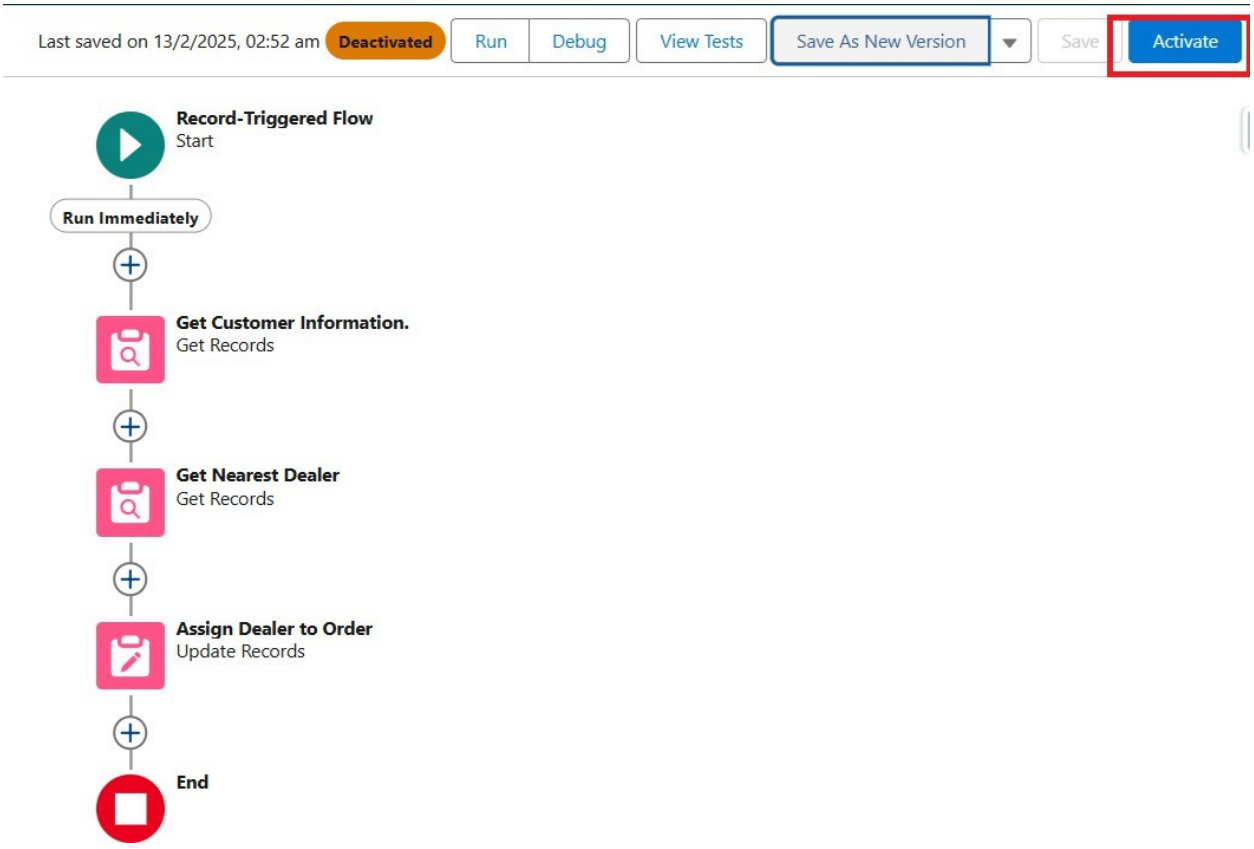


Step 9 : Click Save and Give label Name and Activate Flow.

- Label Name : Auto Assign Dealer
- *How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection
- Select Record(s) to Update : {!Get_Nearest_Dealer}



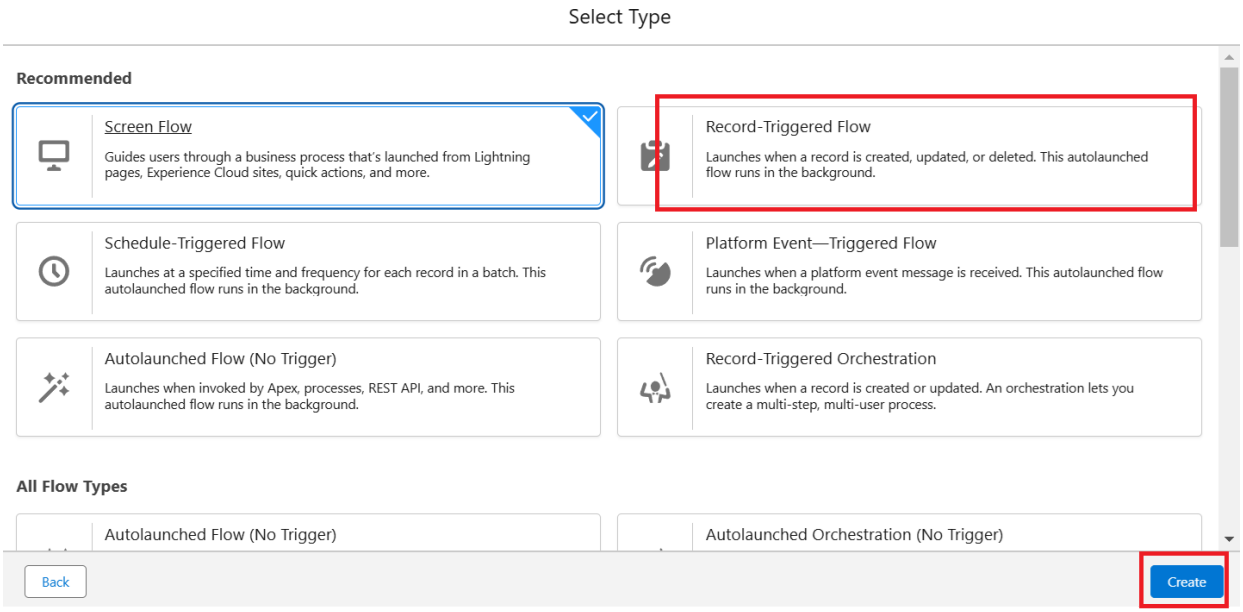
Step 10 : Activate Flow



Record-Trigger Flow Creation :

Creating a record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow and click Create



Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

Set Entry Condition :

- All Conditions Are Met (AND)
- Filed : Status__c
- Operator : Equals
- Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

- Label: Reminder Before Test Drive.
- Time Source: Test_Drive_Date__c
- Offset Number: 1.
- Offset Options: Days Before.

Step 4: Add Action

Select the required action to send a reminder to the customer before the test drive.

Step 5: Configure Email Alert

Create an email alert to notify the customer about the upcoming test drive.

Step 6: Set Conditions

Configure the required conditions to ensure that the reminder is sent at the correct time.

Step 7: Save the Flow

Save the flow with an appropriate name, such as Test Drive Reminder Flow.

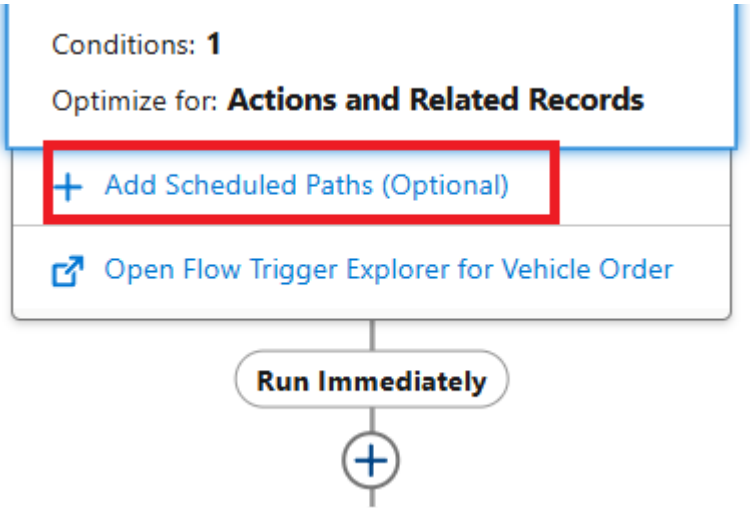
Step 8: Activate the Flow

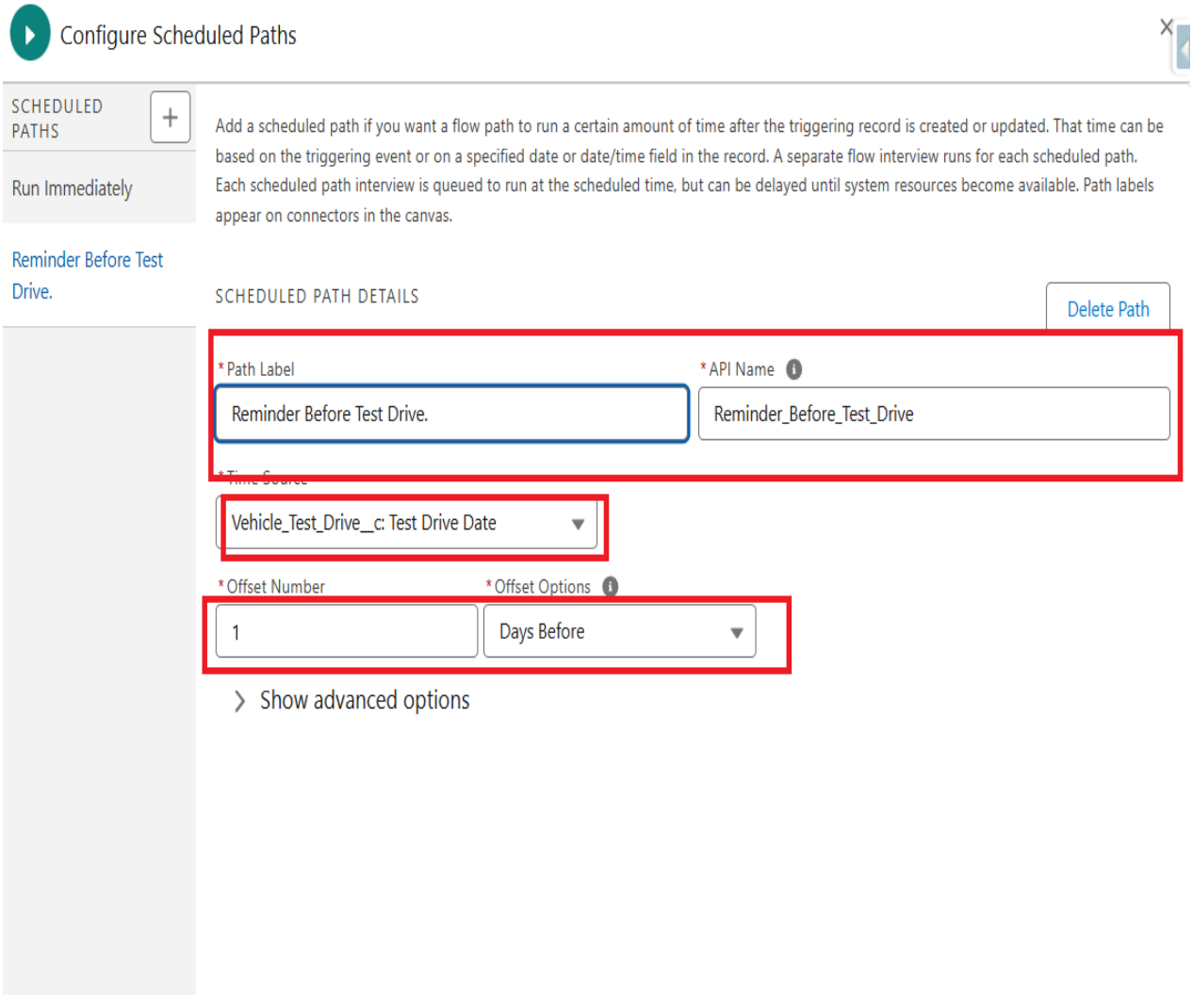
Activate the flow to enable automatic execution.

Step 9: Test the Flow

Create or update a test drive record and verify whether the reminder works as expected.

- Click Done.

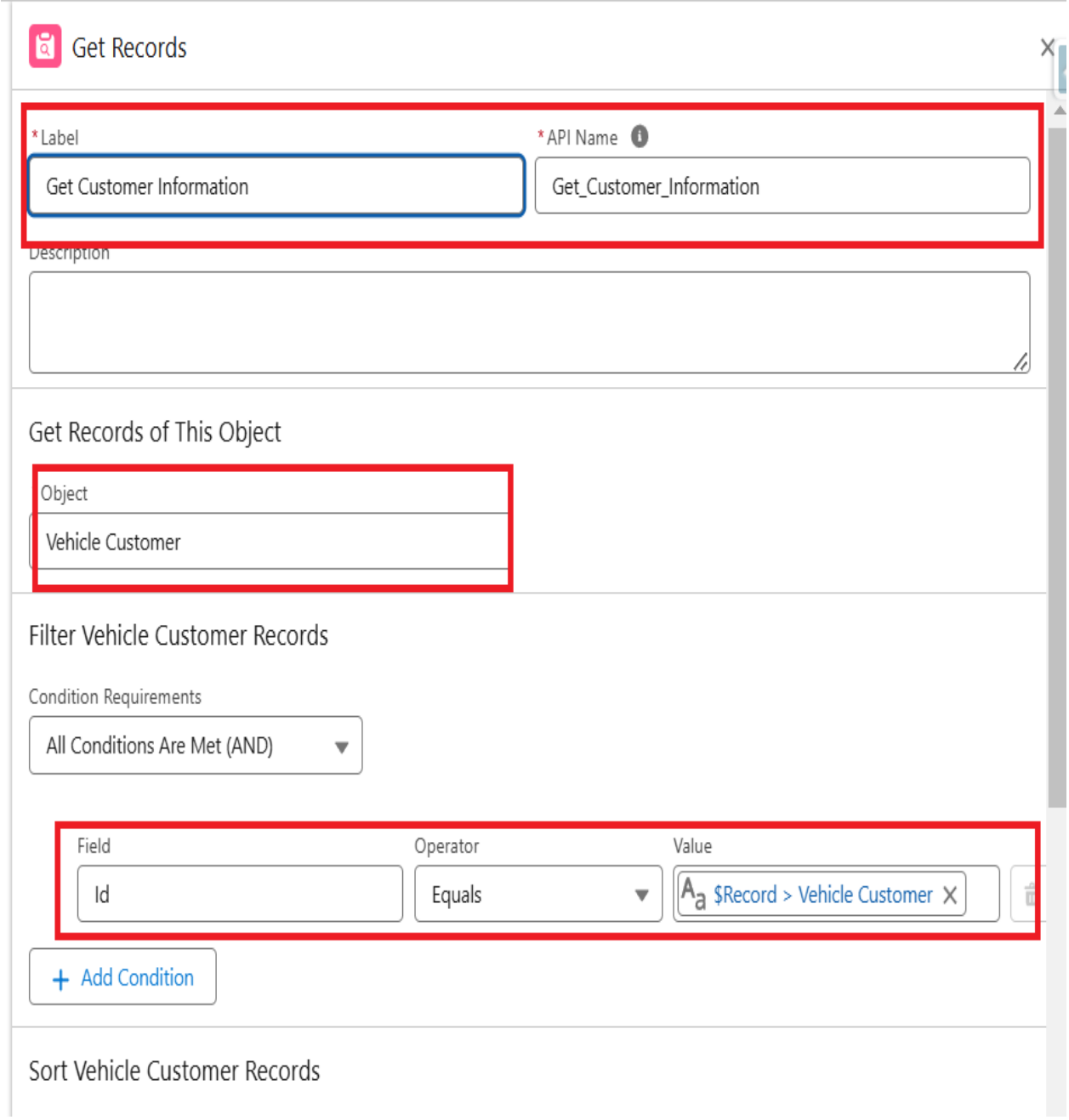

Conditions: 1
Optimize for: Actions and Related Records
Add Scheduled Paths (Optional)
Open Flow Trigger Explorer for Vehicle Order
Run Immediately


Configure Scheduled Paths
SCHEDULED PATHS
Run Immediately
Reminder Before Test Drive.
Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.
SCHEDULED PATH DETAILS
Delete Path
* Path Label
Reminder Before Test Drive.
* API Name
Reminder_Before_Test_Drive
Vehicle_Test_Drive__c: Test Drive Date
* Offset Number
1
* Offset Options
Days Before
Show advanced options

Step:4

Click + Add Element → Get Records.

1. Label: Get Customer Information.
2. Object: Vehicle_Customer__c.
3. Filter Conditions: Id = {!$Record.Customer__c}.
4. How Many Records to Store: Select Only the first record.
5. How to Store Record Data: Choose Automatically store all fields.



Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

```
public class TestDriveReminderEmail {

    public static void sendReminder(String customerEmail) {

        Messaging.SingleEmailMessage email =
            new Messaging.SingleEmailMessage();

        email.setToAddresses(
            new String[] { customerEmail }
        );

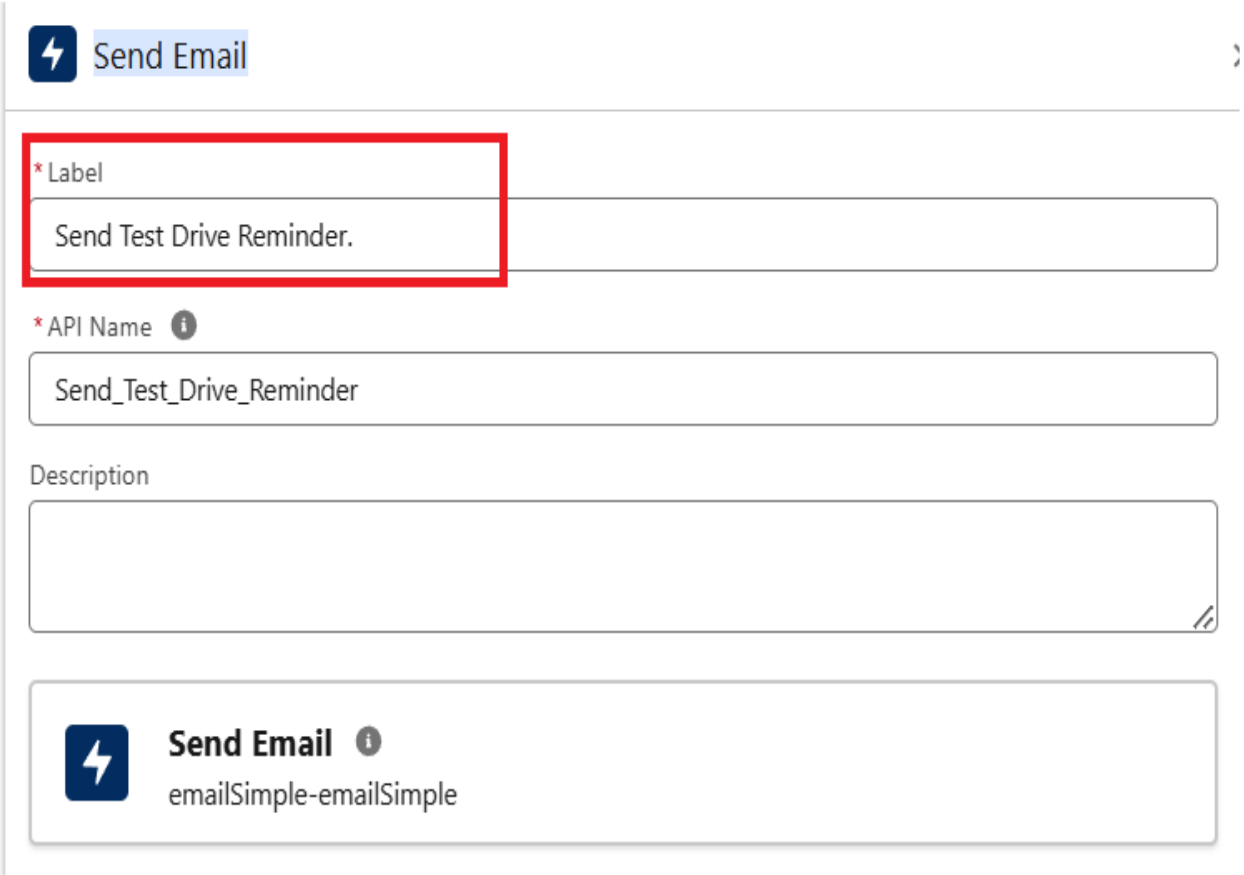
        email.setSubject(
            'Test Drive Reminder - WhatNext Vision Motors'
        );

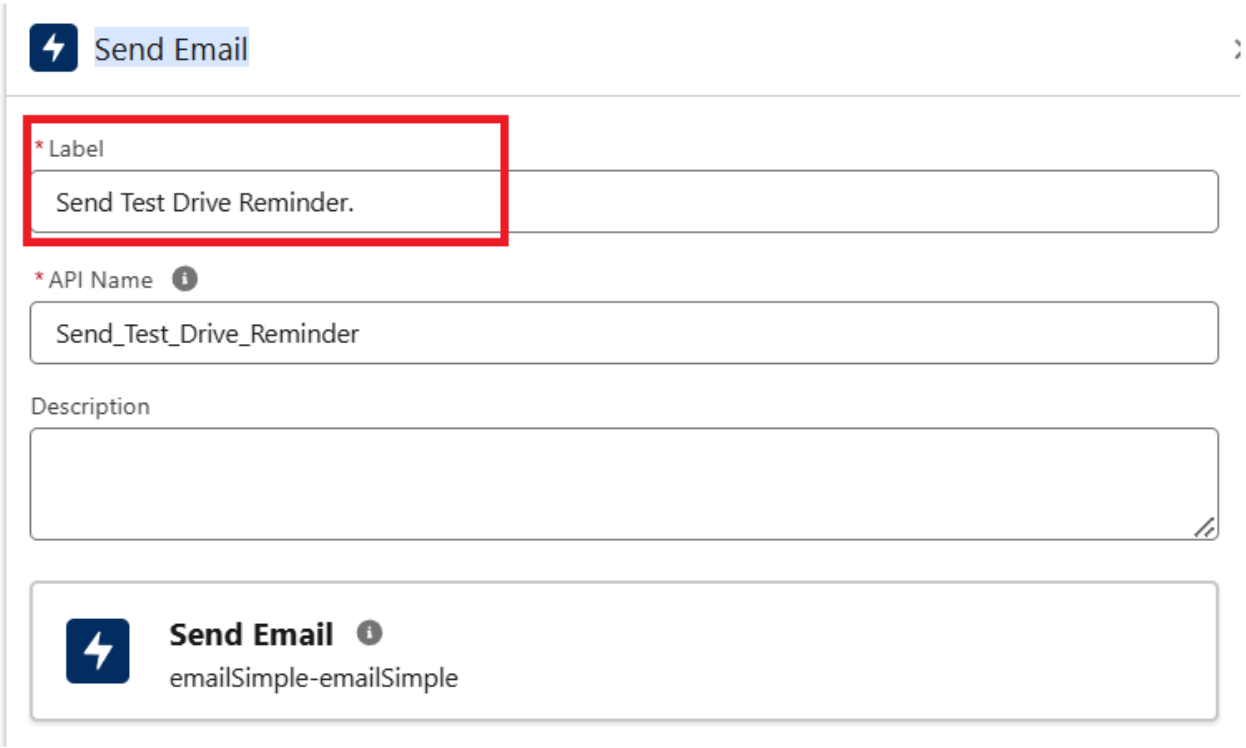
        email.setPlainTextBody(
            'Dear Customer,\n\n' +
            'This is a reminder about your upcoming test drive ' +
            'with WhatNext Vision Motors.\n\n' +
            'Please arrive at the scheduled time.\n\n' +
            'Thank you for choosing WhatNext Vision Motors.'
```

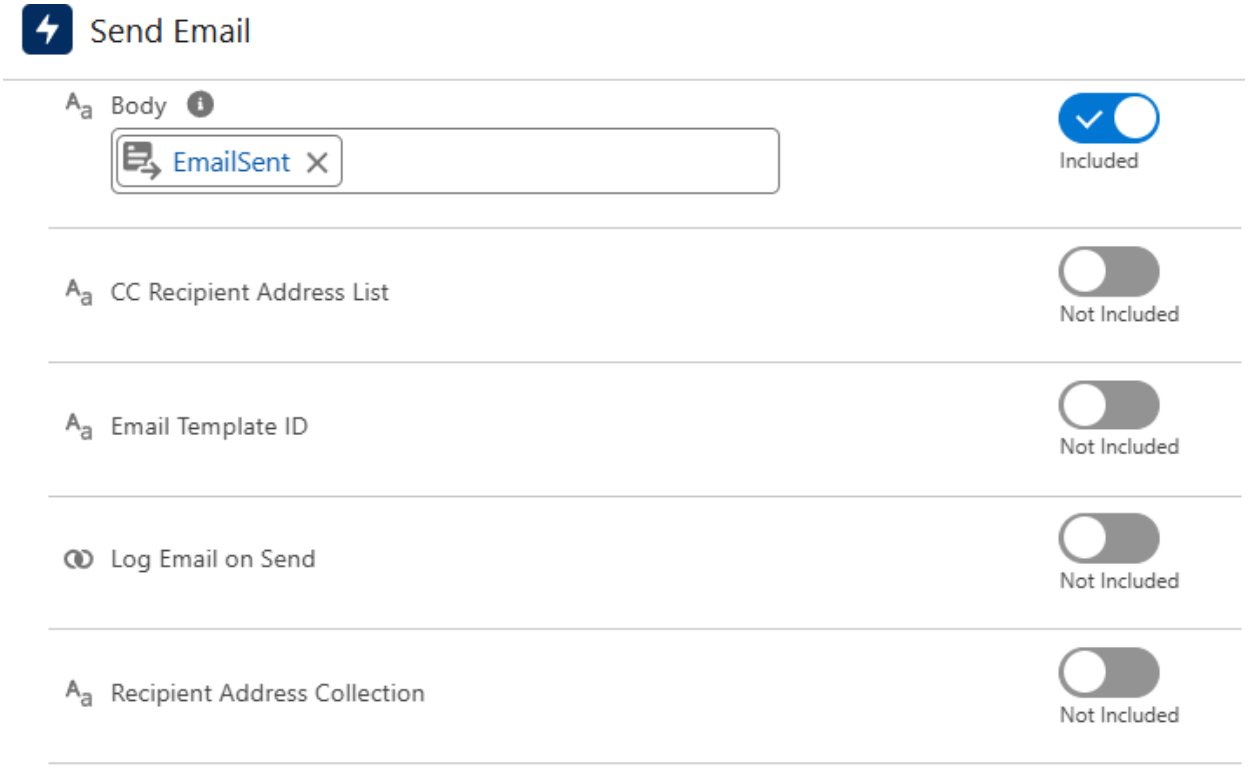
```
        );

        Messaging.sendEmail(
            new Messaging.SingleEmailMessage[] { email }
        );

        System.debug('Test drive reminder email processed.');
    }
}
```

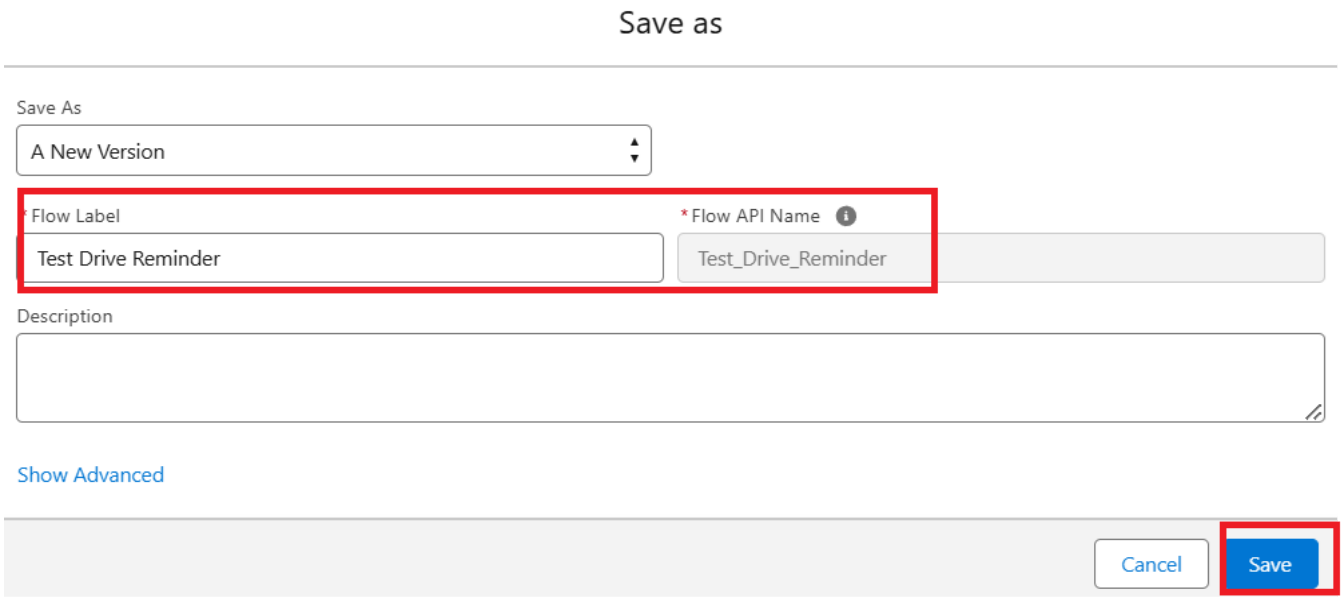



Send Email
* Label
Send Test Drive Reminder.
* API Name
Send_Test_Drive_Reminder
Description
Send Email
emailSimple-emailSimple


Send Email
Body
EmailSent
Included
CC Recipient Address List
Not Included
Email Template ID
Not Included
Log Email on Send
Not Included
Recipient Address Collection
Not Included

Step 6 : Click save

Label Name : Test Drive Reminder



Step 7 : Activate Flow

```
public class VehicleFlowController {

    public static void activateVehicleFlow() {

        Flow.Interview.Vehicle_Management_Flow flow =
            new Flow.Interview.Vehicle_Management_Flow(
                new Map<String, Object>()
            );

        flow.start();

        System.debug('Vehicle Management Flow executed successfully.');
    }
}
```

```
public class VehicleFlowController {

    public static void runVehicleFlow() {

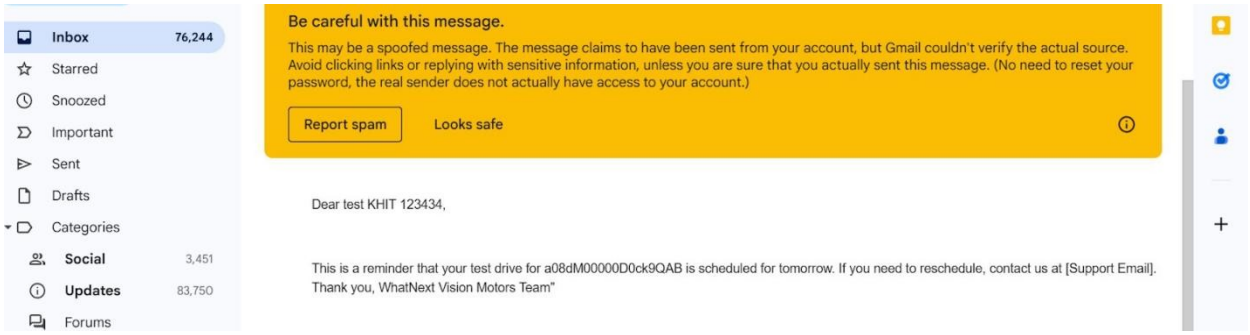
        Map<String, Object> inputVariables =
            new Map<String, Object>();

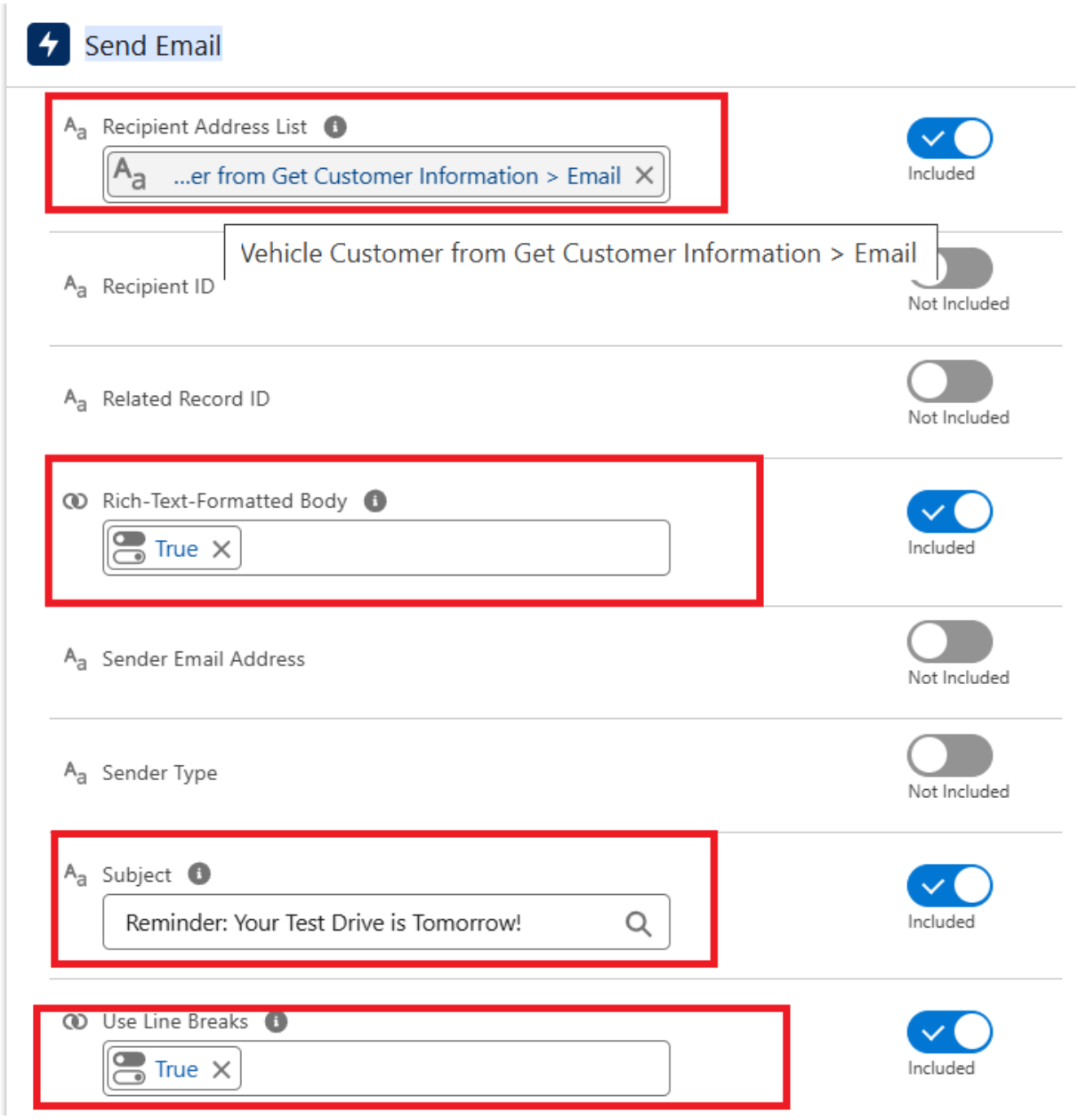
        Flow.Interview.Vehicle_Management_Flow vehicleFlow =
            new Flow.Interview.Vehicle_Management_Flow(
                inputVariables
            );

        vehicleFlow.start();

        System.debug('WhatNext Vision Motors');
        System.debug('Vehicle Management Flow Started');
        System.debug('Flow executed successfully');
    }
}
```



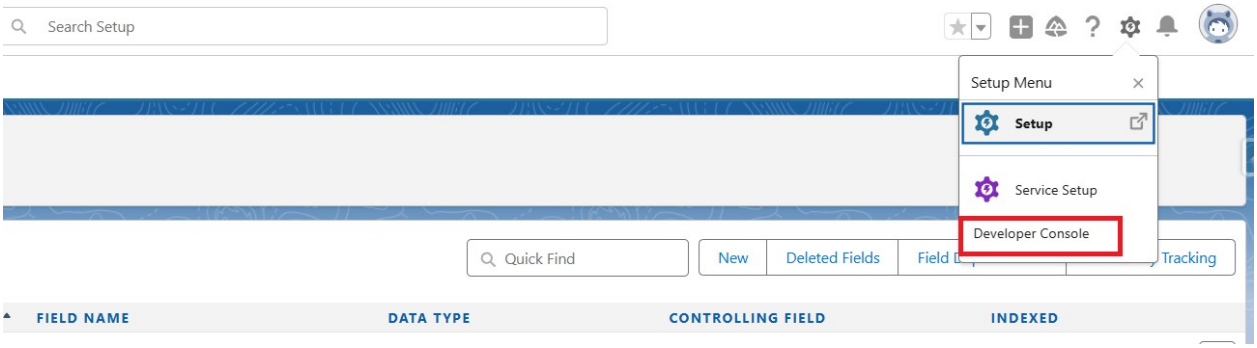

Inbox 76,244
Starred
Snoozed
Important
Sent
Drafts
Categories
Social 3,451
Updates 83,750
Forums
Be careful with this message.
This may be a spoofed message. The message claims to have been sent from your account, but Gmail couldn't verify the actual source. Avoid clicking links or replying with sensitive information, unless you are sure that you actually sent this message. (No need to reset your password, the real sender does not actually have access to your account.)
Report spam
Looks safe
Dear test KHIT 123434,
This is a reminder that your test drive for a08dM00000D0ck9QAB is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email].
Thank you, WhatNext Vision Motors Team"


Send Email
Recipient Address List
...er from Get Customer Information > Email
Included
Vehicle Customer from Get Customer Information > Email
Recipient ID
Not Included
Related Record ID
Not Included
Rich-Text-Formatted Body
True
Included
Sender Email Address
Not Included
Sender Type
Not Included
Subject
Reminder: Your Test Drive is Tomorrow!
Included
Use Line Breaks
True
Included

Milestone 7: Apex and Batch Class

Create Apex and Trigger Batch Jobs :

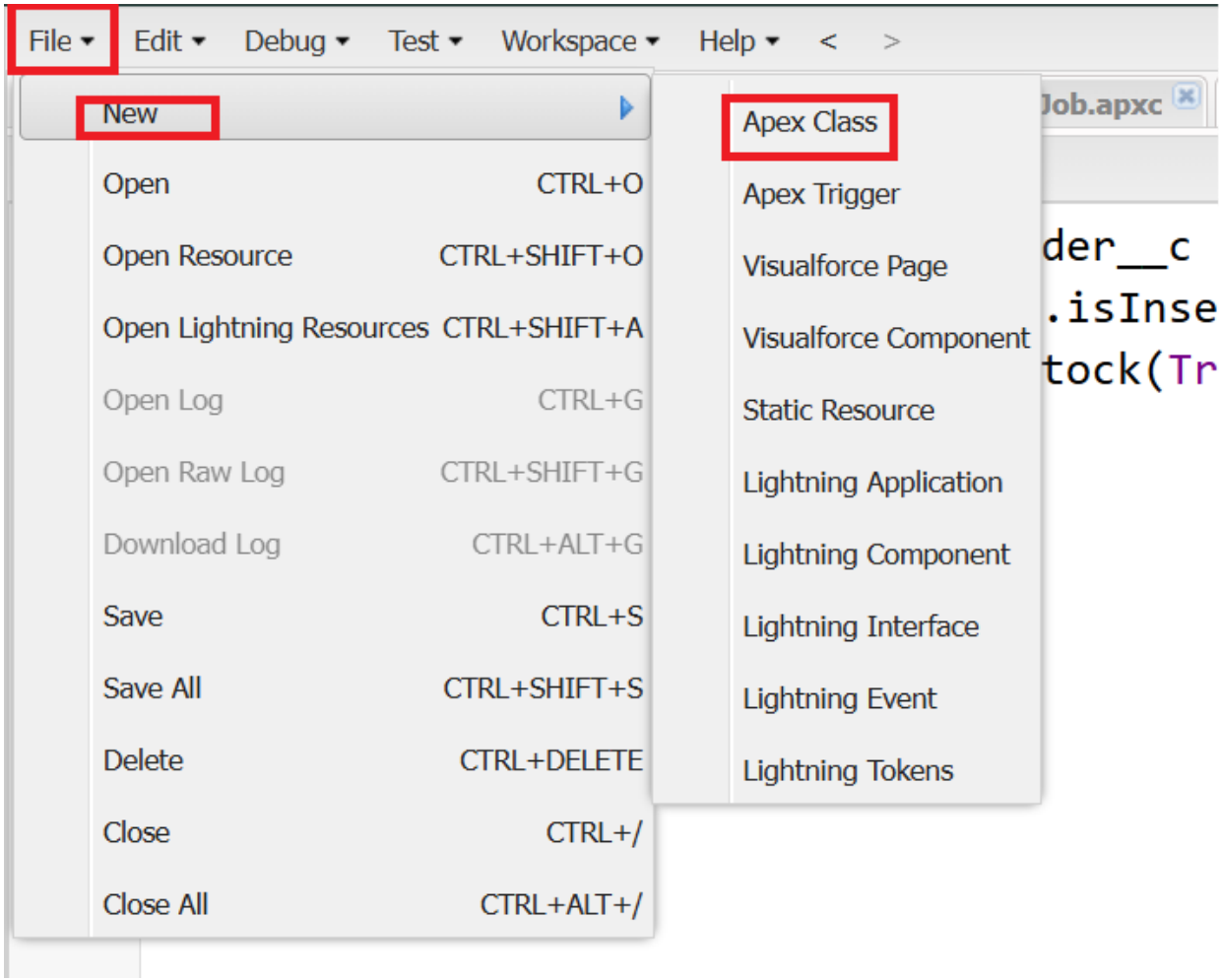
Step 1 : Click Developer Console From Gear icon



Step 2 : click File then Select New then Select Apex.

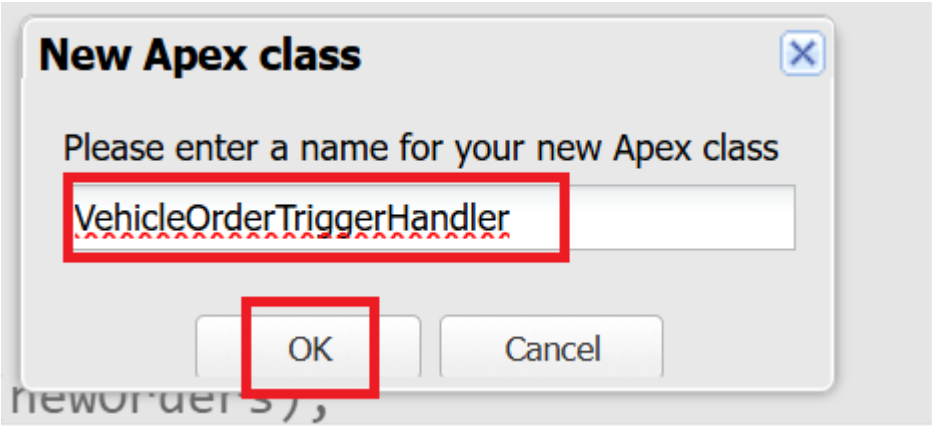
```
public class VehicleManagement {

public static void displayDetails() {
System.debug('WhatNext Vision Motors');
System.debug('Vehicle Management System');
System.debug('Vehicle Name: Tata Nexon');
System.debug('Vehicle Status: Available');
System.debug('Vehicle Stock: 10');
}
}
```



Step 3 : Give Name For Apex Class

- Class Name : VehicleOrderTriggerhandler



Step 4 : Write Apex Code

```
public class VehicleManagement {

    public static void displayVehicle() {

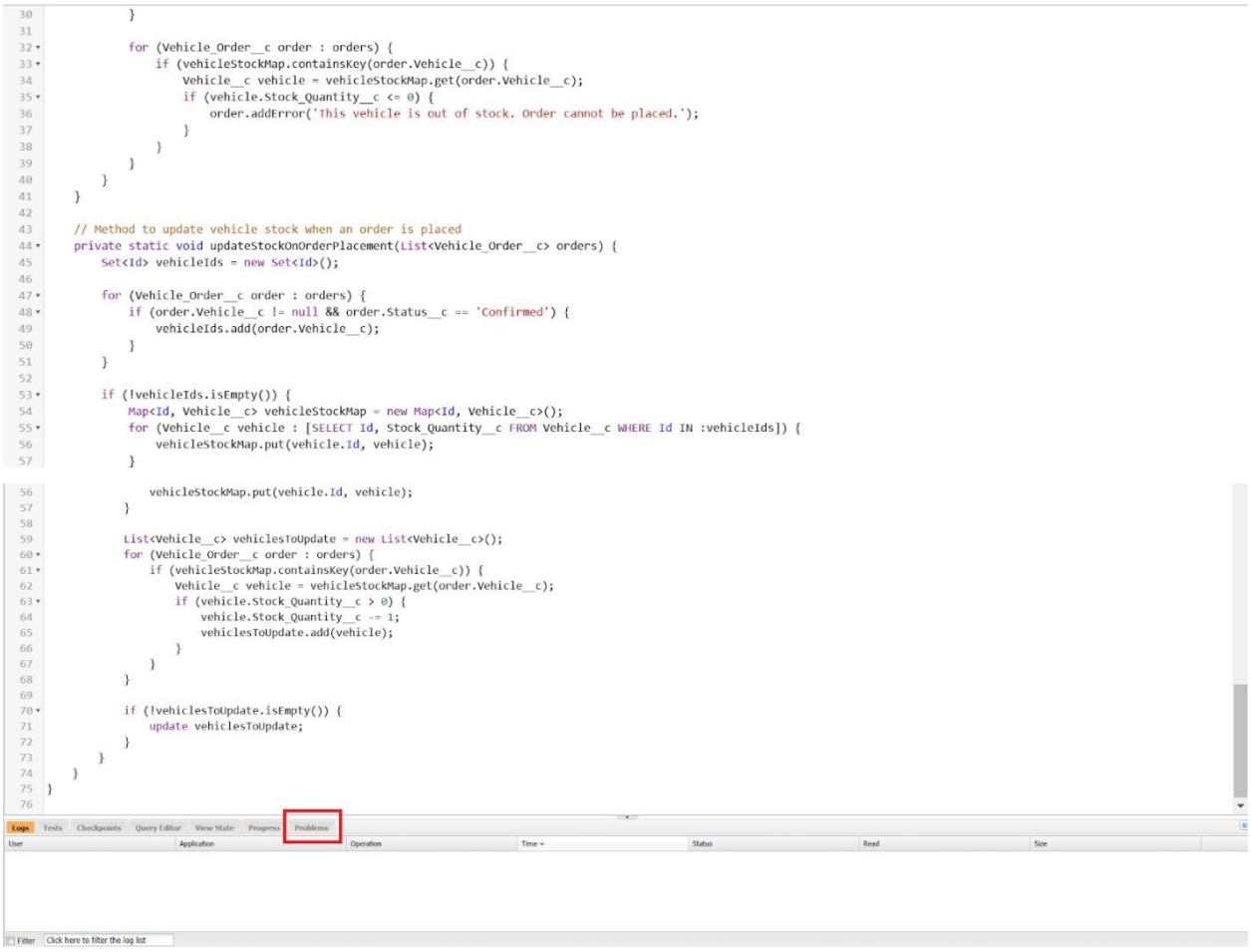
        String vehicleName = 'Tata Nexon';
        Decimal vehiclePrice = 850000;
        Integer stock = 10;

        System.debug('WhatNext Vision Motors');
        System.debug('Vehicle Name: ' + vehicleName);
        System.debug('Vehicle Price: Rs. ' + vehiclePrice);
        System.debug('Available Stock: ' + stock);

        if (stock > 0) {
            System.debug('Vehicle is Available');
        } else {
            System.debug('Vehicle is Not Available');
        }
    }
}

Map<String, Object> inputVariables =
        new Map<String, Object>();

    Flow.Interview.Vehicle_Management_Flow vehicleFlow =
        new Flow.Interview.Vehicle_Management_Flow(
            inputVariables
```



Source Code :

```
public class VehicleOrderTriggerHandler
    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
    Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter,
    Boolean    isInsert, Boolean isUpdate) {

    if (isBefore) {

      if      (isInsert      ||      isUpdate)      {
        preventOrderIfOutOfStock(newOrders
        );
      }

    }
```

```
    if (isAfter) {

        if (isInsert || isUpdate) {
            updateStockOnOrderPlacement(newOrders);
        }

    }

}

// Method to prevent orders when the vehicle is out of stock

private static void preventOrderIfOutOfStock(List<Vehicle_Order__c>
    orders) { Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order :
        orders) { if (order.Vehicle__c
        != null) {
            vehicleIds.add(order.Vehicle__c);

        }

    }


     Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle
    if (vehicle.Stock_Quantity__c > 0) {
                vehicle.Stock_Quantity_____c -= 1;
                vehiclesToUpdate.add(vehicle);
```

```
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM
Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle =
                    vehicleStockMap.get(order.Vehicle__c); if
                    (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }
    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__
        c> orders) { Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
if (order.Vehicle__c != null && order.Status__c == 'Confirmed')
```

```
{ vehicleIds.add(order.Vehicle__c);
            }
public class WhatNextVisionMotors {

public static void main(String[] args) {

String vehicleName = "Tata Nexon";
double vehiclePrice = 850000;
int availableStock = 10;

System.out.println("WhatNext Vision Motors");
System.out.println("----------------------");
System.out.println("Vehicle Name: " + vehicleName);
System.out.println("Vehicle Price: Rs. " + vehiclePrice);
System.out.println("Available Stock: " + availableStock);

if (availableStock > 0) {
System.out.println("Vehicle Status: Available");
} else {
System.out.println("Vehicle Status: Not Available");
}
}
```

```
    private static void updateStockOnOrderPlacement(List<Vehicle_Order_
      c> orders) { Set<Id> vehicleIds = new Set<Id>();
      for (Vehicle_Order__c order : orders) {
```

```
if (!vehicleIds.isEmpty()) {

    Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

    for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM
Vehicle__c WHERE Id IN :vehicleIds]) {

      vehicleStockMap.put(vehicle.Id, vehicle);

    }

    List<Vehicle_____c> vehiclesToUpdate = new
    List<Vehicle__c>(); for (Vehicle_Order__c order :
    orders) {
      if (vehicleStockMap.containsKey(order.Vehicle__c)) {

        Vehicle_____c vehicle =
        vehicleStockMap.get(order.Vehicle_____c); if
        (vehicle.Stock_Quantity__c > 0) {
          vehicle.Stock_Quantity_____c -= 1;
          vehiclesToUpdate.add(vehicle);
        }

      }

    }

    if (!
      vehiclesToUpdate.isEm
      pty()) { update
      vehiclesToUpdate;
    }

  }

}
```

```
        }
# WhatNext Vision Motors
# Vehicle Management System

vehicles = []

def add_vehicle():
    vehicle_id = input("Enter Vehicle ID: ")
    vehicle_name = input("Enter Vehicle Name: ")
    price = float(input("Enter Vehicle Price: "))

    vehicle = {
        "ID": vehicle_id,
        "Name": vehicle_name,
        "Price": price
    }

    vehicles.append(vehicle)
    print("Vehicle added successfully!")

def view_vehicles():
    if not vehicles:
        print("No vehicles available.")
    else:
        print("\n--- Available Vehicles ---")
        for vehicle in vehicles:
            print("ID:", vehicle["ID"])
            print("Name:", vehicle["Name"])
            print("Price: Rs.", vehicle["Price"])
            print("--------------------")
```

```
def search_vehicle():
    name = input("Enter vehicle name to search: ")
    found = False

    for vehicle in vehicles:
        if vehicle["Name"].lower() == name.lower():
            print("Vehicle Found:", vehicle)
            found = True

    if not found:
        print("Vehicle not found.")

while True:
    print("\n=== WhatNext Vision Motors ===")
    print("1. Add Vehicle")
    print("2. View Vehicles")
    print("3. Search Vehicle")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_vehicle()
    elif choice == "2":
        view_vehicles()
    elif choice == "3":
        search_vehicle()
    elif choice == "4":
        print("Thank you for using WhatNext Vision Motors!")
        break
    else:
```

```
        print("Invalid choice. Try again.")


print("WhatNext Vision Motors")

vehicles = ["Tata Nexon", "Hyundai Creta", "Kia Seltos"]

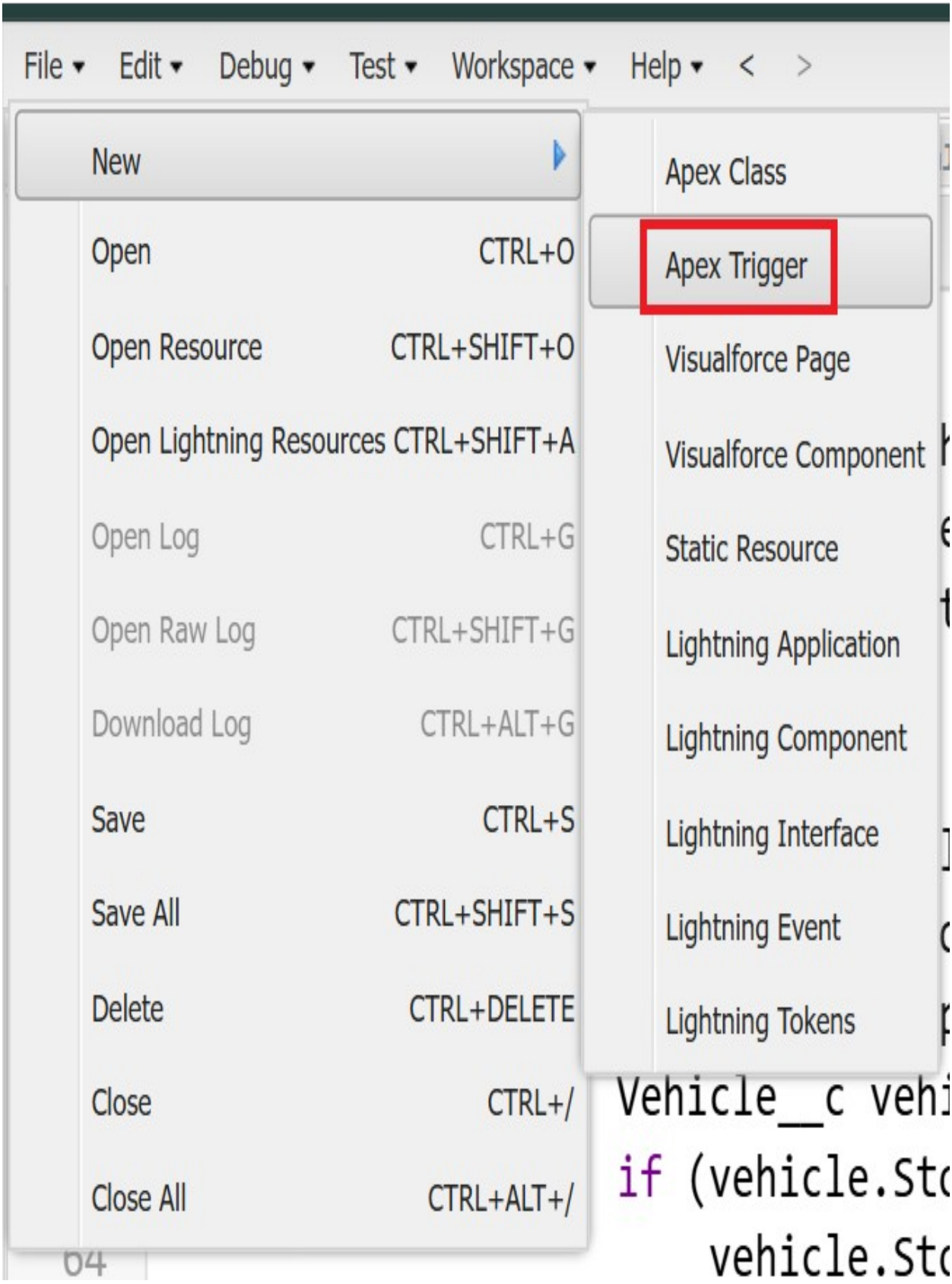
print("Available Vehicles:")

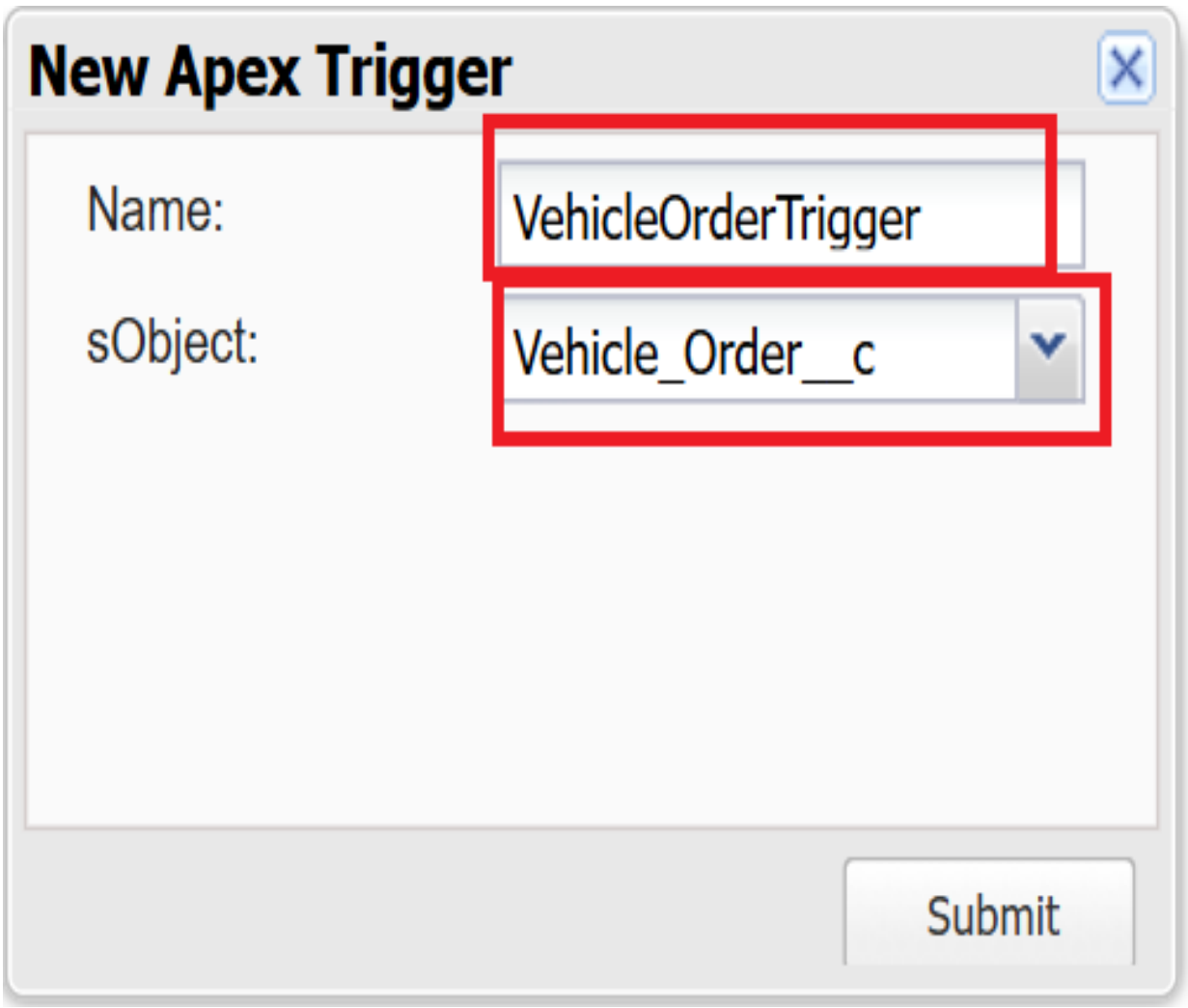
for vehicle in vehicles:
    print(vehicle)

name = input("Enter vehicle name to search: ")

if name in vehicles:
    print("Vehicle Available")
else:
    print("Vehicle Not Available")

if choice == "1":
        add_vehicle()
    elif choice == "2":
        view_vehicles()
    elif choice == "3":
        search_vehicle()
    elif choice == "4":
        print("Thank you for using WhatNext Vision Motors!")
```


File
Edit
Debug
Test
Workspace
Help
New
Open CTRL+O
Open Resource CTRL+SHIFT+O
Open Lightning Resources CTRL+SHIFT+A
Open Log CTRL+G
Open Raw Log CTRL+SHIFT+G
Download Log CTRL+ALT+G
Save CTRL+S
Save All CTRL+SHIFT+S
Delete CTRL+DELETE
Close CTRL+/
Close All CTRL+ALT+/
Apex Class
Apex Trigger
Visualforce Page
Visualforce Component
Static Resource
Lightning Application
Lightning Component
Lightning Interface
Lightning Event
Lightning Tokens
Vehicle__c vehi
if (vehicle.Sto
vehicle.Sto



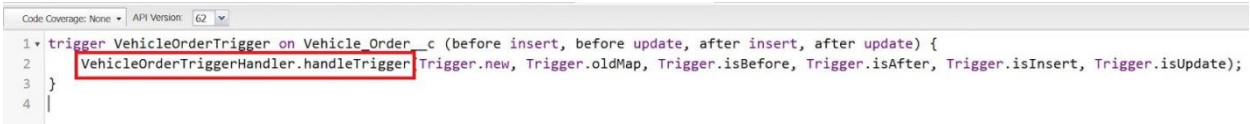
Step 6 : Call Apex Class in Trigger

Class Source Code:

trigger VehicleOrderTrigger on Vehicle_Order___c (before insert,before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);

}



Step 7 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.

```
global class VehicleBatchJob implements Database.Batchable<SObject> {

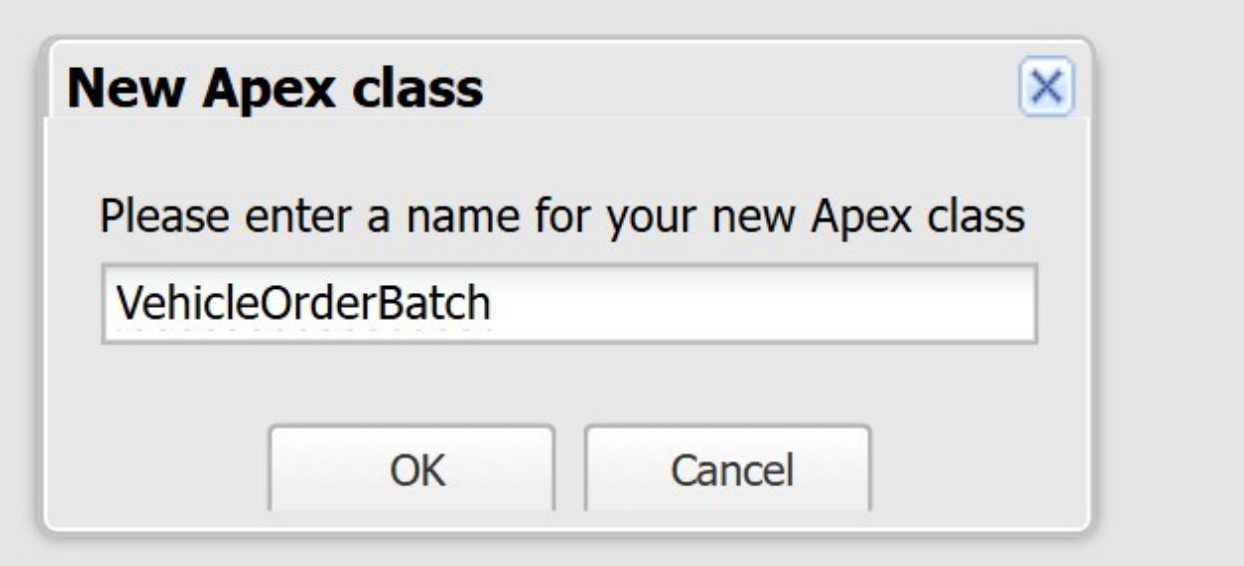
    global Database.QueryLocator start(
        Database.BatchableContext bc
    ) {
        return Database.getQueryLocator(
            'SELECT Id, Name FROM Account'
        );
    }

    global void execute(
        Database.BatchableContext bc,
        List<Account> records
    ) {
        for (Account acc : records) {
            acc.Description = 'Vehicle Management Record';
        }

        update records;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle Batch Job Completed');
```

}



Code Coverage: None API Version: 62

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

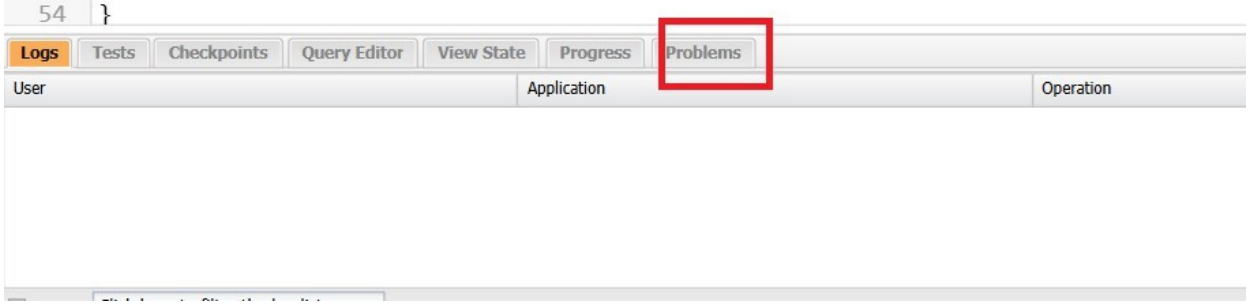
            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!ordersToUpdate.isEmpty()) {
                update ordersToUpdate;
            }

            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```



Source Code:

global class VehicleOrderBatch implements Database.Batchable<sObject> { global Database.QueryLocator start (Database.BatchableContext bc) { return Database.getQueryLocator([

SELECT Id, Status_c,

Vehicle_____c

FROM Vehicle_Order_c

WHERE Status_c

```
'Pending'

]);

}

global void execute(Database.BatchableContext bc, List<Vehicle_Order_c>
orderList) { Set<Id> vehicleIds = new Set<Id>();
for (Vehicle_Order_c order
orderList) { if
(order.Vehicle_c != null)
{ vehicleIds.add(order.Vehicle
c);
}

}

if (!vehicleIds.isEmpty()) {

Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehiclec>();

for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c
WHERE Id IN vehicleIds]) {

vehicleStockMap.put(vehicle. Id, vehicle);

}

List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>();
List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

for (Vehicle_Order_c order orderList) {

if (vehicleStockMap.containsKey(order. Vehicle_
c)) { Vehicle_c vehicle =
vehicleStockMap.get(order. Vehicle_____c); if
```

```
(vehicle.Stock_Quantity___c > 0) {
order.Status__c = 'Confirmed'

ersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);
}

}

}

if (!
ordersToUpdate.isEmpt
y()) { update
ordersToUpdate;
}

if (!vehicles
ToUpdate.isEmpty()) { update
vehiclesToUpdate;
}

}

global void finish(Database.BatchableContext bc)
{ System.debug('Vehicle order batch job completed.');
}


if (!vehicles
ToUpdate.isEmpty()) { update
vehiclesToUpdate;
}
```

```
ersToUpdate.add(vehicle);
        }

        }

        }

        if                    (!
        ordersToUpdate.isEmpt
        y())      {      update
        ordersToUpdate;
        }

        if                    (!vehicles
        ToUpdate.isEmpty()) { update
        vehiclesToUpdate;
        }

        }

        global     void     finish(Database.BatchableContext     bc)
        { System.debug('Vehicle order batch job completed.');
        }

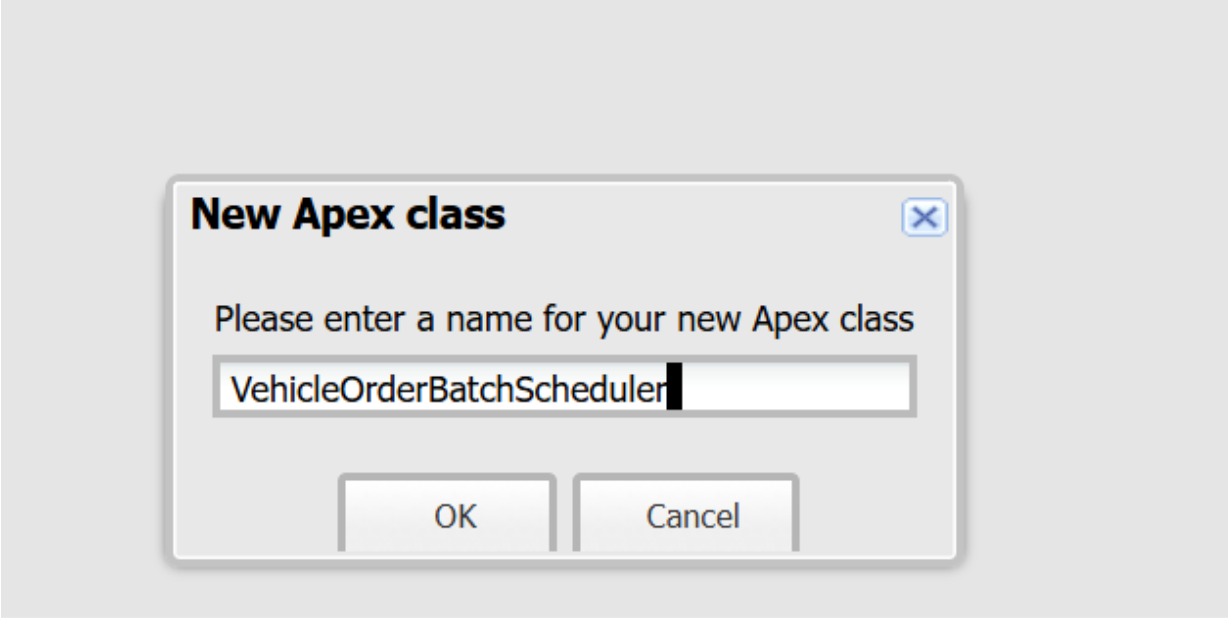
        }

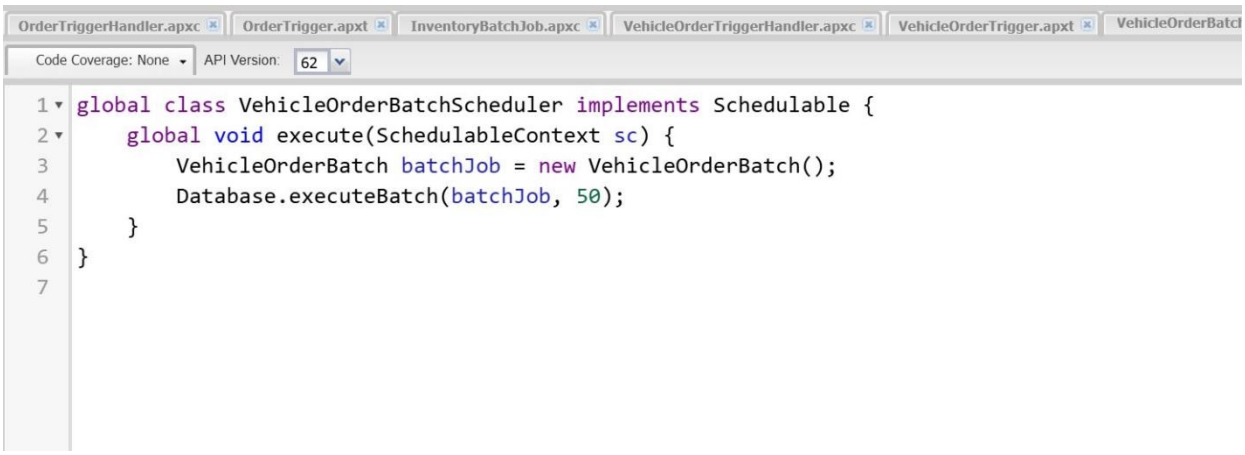
        if (!vehicleIds.isEmpty()) {

        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehiclec>();

        for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c
        WHERE Id IN vehicleIds]) {

        vehicleStockMap.put(vehicle. Id, vehicle);
```



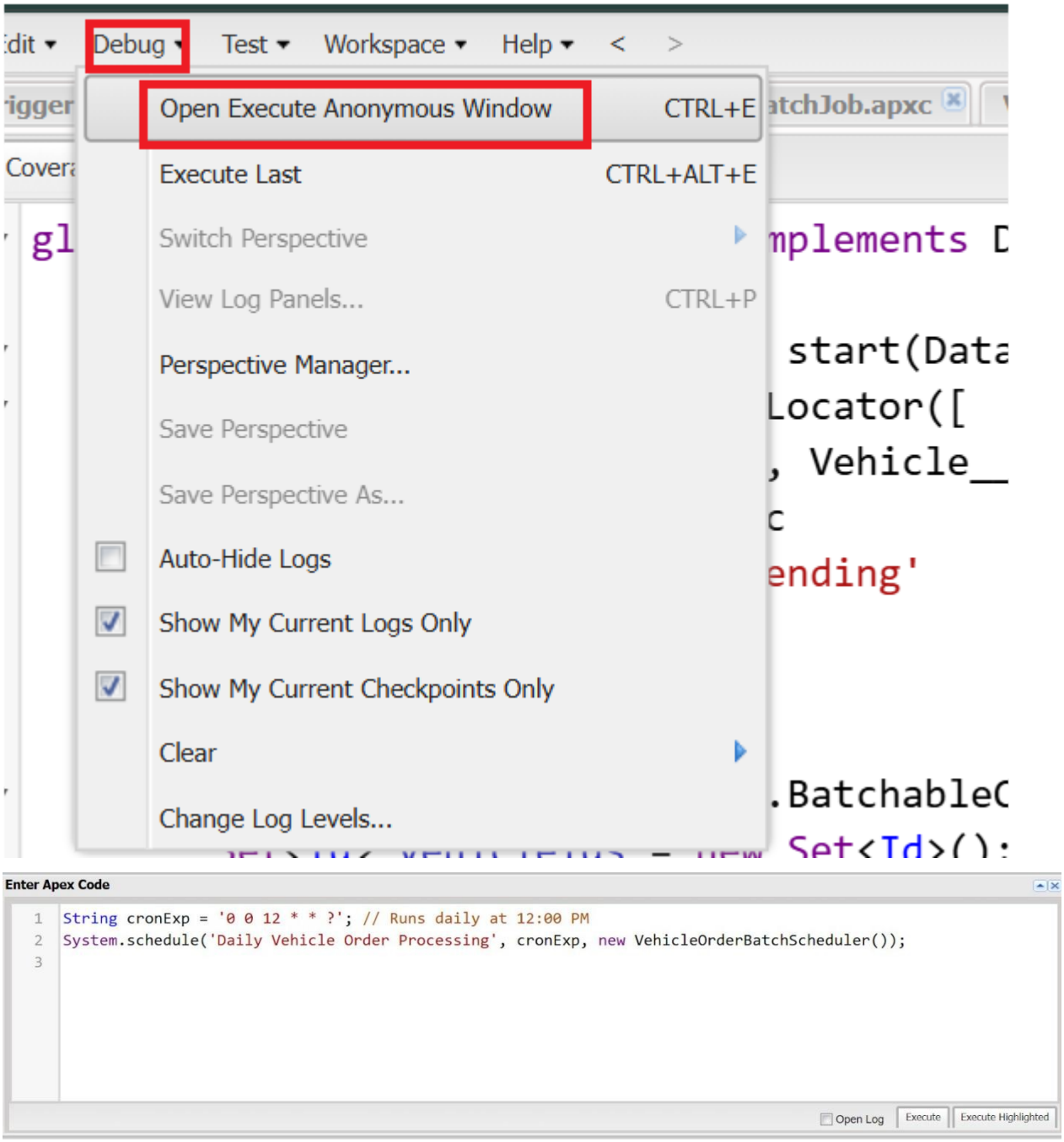


Source Code :

```
global class VehicleOrderBatchScheduler implements
  Schedulable { global void execute(SchedulableContext sc)
  {
    VehicleOrderBatch batchJob = new VehicleOrderBatch();
    Database.executeBatch(batchJob, 50); // 50 is the batch size
  }

}
```

Step 10 : **Schedule the Batch Job** To run the batch job **every night at midnight** :

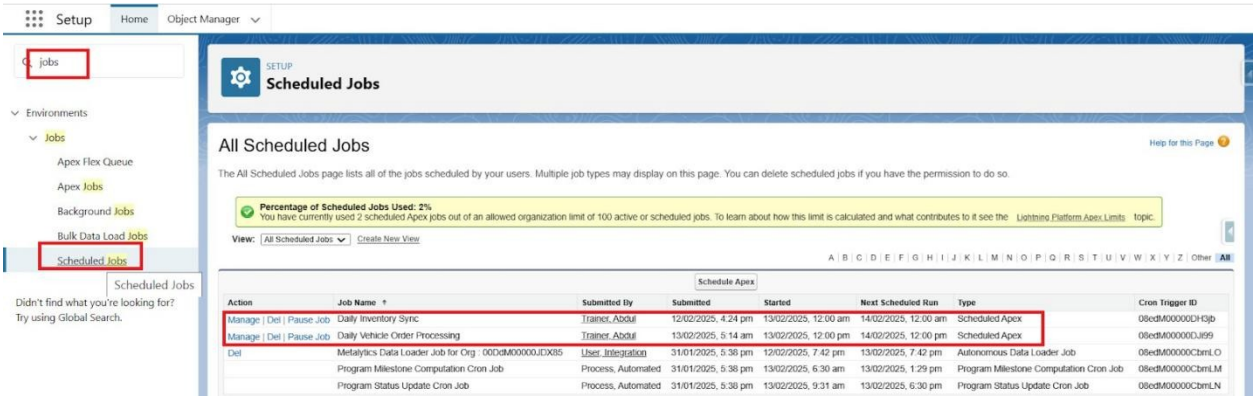


Source Code :

String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running



6. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

Functional and performance testing are important phases in the development of the WhatNext Vision Motors application. Functional testing ensures that all application features, such as vehicle management, inventory tracking, customer management, and order processing, work according to the specified requirements. It verifies that users can perform the required operations correctly and that the system produces the expected results. Performance testing evaluates the application's speed, responsiveness, stability, and behavior under different workloads. It helps identify delays and performance issues when multiple users access the system or when large amounts of data are processed.

7. ADVANTAGES & DISADVANTAGES

Advantages

- Centralized data
- Automated workflows
- Real-time reporting

Disadvantages

- Efficient Vehicle Management
- Inventory Tracking
- Customer Management
- Time Saving

8. CONCLUSION:

The Salesforce CRM implementation at WhatNext Vision Motors successfully modernizes vehicle ordering and dealership management. Automation improves efficiency, accuracy, and customer experien